

“University Management System”

A

Project Report

*Submitted in partial fulfillment of the requirement for the award of
Degree of*

Bachelor of Computer Applications

Submitted to

**452 – Catalyst Institute of Management and
Advance Global Excellence**



Submitted By
ASHISH SINHA
Registration No.<202245200034>
BCA-3RD Year
Session-2022-2025

Under Joint Guidance of
Prof. Ravi Kumar Soni &
Prof. Niraj Kumar Singh
[Asst. Professor]
Computer Science &
Information Technology



**<Catalyst Institute of Management and Advance
Global Excellence>**
Department of Computer Science & Information Technology
Session 2022-25

CERTIFICATE

This is to certify that the work embodies in this project entitled, **“University management system”**, being submitted by **Ashish Sinha** (202245200034) in partial fulfillment of the requirement for the award of **“Bachelor of Computer Applications”** to **PATLIPUTRA UNIVERSITY, PATNA** during the academic year 2021-24 is a record of bonafide piece of work, carried out by him/her under our/my supervision and guidance in the **“Department of Computer Science & information Technology”**, <Catalyst Institute of Management and Advance Global Excellence>.



Prof. Neeraj Agrawal
Director

Er. Amit Kumar Shukla
Head of Department-IT

Prof. Ravi Soni
Assistant Professor-IT



< Catalyst Institute of Management and Advance Global Excellence >

Department of Computer Science & Information Technology

Session 2022-25

Date:

APPROVAL CERTIFICATE

The project report entitled “**University management system**”, being submitted by

Ashish Sinha (202245200034) has been examined by us and is hereby approved for the award of degree “**Bachelor of Computer Applications**”, for which it has been submitted. It is understood that by this approval the undersigned do not necessarily endorse or approve any statement made, the opinion expressed or conclusion drawn therein, but approve the project only for the purpose for which it has been submitted.



(Internal Examiner)

(External Examiner)

Date:

Date:



Catalyst Institute of Management and Advance Global Excellence
Department of Computer Science & Information Technology

Session 2022-25

Date:.....

DECLARATION

I, **Ashish Sinha** hereby declare that the work, which is being presented in the project report entitled “partial fulfillment of the requirements for the award of degree of “**Bachelor of Computer Applications**” submitted in the department of **Computer Science & Information Technology** (Catalyst Institute of Management and Advance Global

Excellence) is an authentic record of my own work carried under the joint guidance of **Prof. Niraj Kumar Singh Sir and Prof. Ravi Kumar Soni Sir**. I have not submitted the matter embodied in this report for the award of any other degree.

ASHISH SINHA

Registration No: **202245200034**

BCA 3RD Year

Session-2022-2025

ACKNOWLEDGEMENT

“A journey is easier when you travel together. Interdependence is certainly more valuable than independence.”

I would like to thanks, **Prof. Niraj Kumar Singh Sir and Prof. Ravi Soni Sir**, for providing regular guidance and insight into my project work. I also thank them for all advice he has given me in the past year, and for always having time for me, whenever I needed.

I give special thanks to **Er. Amit Kumar Shukla Sir, Prof. Anjesh Sir and Prof. Raju Upadhyay Sir** for always being willing to help find solutions to any problems I had with my work.

“The completion of any project depends upon the cooperation, coordination, and combined efforts of several resources of knowledge, inspiration, and energy”.

I also extend my deepest gratitude to **Director, CIMAGE GROUP OF INSTITUTIONS, PATNA, Prof. Neeraj Agrawal Sir, Dean CIMAGE GROUP OF INSTITUTIONS, PATNA, Prof. Neeraj Kumar Sir and Centre Head Mrs. Megha Agrawal Mam** for providing all the necessary facilities and true encouraging environment to bring out the best in my endeavors.

I express my gratitude and thanks to all the staff members of Computer Science departments for their sincere cooperation in furnishing relevant information to complete this Project well in time successfully.

I extend a special word to my friends, who have been a constant source of inspiration throughout my project work.

Lastly but not least I must express my cordial thank to my parent and family members who gave me the moral support without that it is impossible to complete my project work. With this note, I thank everyone for the support.

(Ashish Sinha)

Registration No: **202245200034**

BCA 3rd Year

Session-2022-2025

REPORT	0
INTRODUCTION.....	5
OBJECTIVE	6
PROJECT CATEGORY JAVA	7
ADVANTAGES OF JAVA	11
TOOLS/PLATFORM required	13
HARDWARE & SOFTWARE REQUIREMENTS	13
PROBLEM DEFINITION	14
REQUIREMENT SPECIFICATIONS	14
PROJECT PLANNING & SCHEDULING	14
GANTT CHART	16
PERT CHART	18
SYSTEM ANALYSIS	1
IDENTIFICATION OF NEED	1
FEASIBILITY STUDY	6
<i>TECHNICAL FEASIBILITY:</i>	7
<i>ECONOMICAL FEASIBILITY:</i>	7
<i>OPERATIONAL FEASIBILITY:</i>	8
ANALYSIS.....	10
DFD	10
MODULE DESCRIPTION	16
DATA STRUCTURE Snap shot	17
LIST OF REPORTS	22
SCOPE AND FUTURE ENHANCEMENT	23
IMPLEMENTATION METHODOLOGY	23
TESTING.....	25
LIMITATIONSANDFURTHER ENHANCEMENTS	28
SECURITY MECHANISM	29

SAMPLE PROJECT

INTRODUCTION

UNIVERSITY MANAGEMENT SYSTEM (UMS) is a flagship product of Easy Solution which covers all aspects of Universities, Colleges or Schools. UMS covers every minute aspects of a universities work flow and integrates all processes with user friendly interface. With hundreds of satisfied customers UMS is first choice of several state, governments/semi- government universities and institutions. UMS is an outcome of hard work done by our expert technical team in supervision of several renowned educationists which includes Controller of examination, faculties. UMS is a rare combination of experience and precision. UMS streamline path of information flow in organization by taking care of following departments:

- Fee Department□
- Examination Department□
- Attendance□
- Faculty information portal□
- Student information portal□

OBJECTIVE

The motive of the project entitled “**University management system**” is to maintain all the record and activities taken place in the administrative department of the university and the colleges

The objectives of this system:

- Drive operational efficiency.□
- Self-service systems with simple to use with little or no training.□
- Elimination of duplicate data entry processes.□
- Integrated with Online Application workflow with unified data model.□
- Monitoring and decision support system.□
- Automation of all the Academic / Examination / Administration operations.□
- Ease and accuracy of reporting.□

Scope:

This project deals with the various functioning in College management process. The main idea is to implement a proper process to system. In our Page 9 existing system contains a many operations registration, student search, fees, attendance, exam records, performance of the student etc. All these activity takeout manually by administrator

PROJECT CATEGORY JAVA

The Java is a programming language originally developed by James Gosling at Sun Microsystems (which is now a subsidiary of Oracle Corporation) and released in 1995 as a core component of Sun Microsystems' Java platform. The language derives much of its syntax from C and C++ but has a simpler object model and fewer low-level facilities. Java applications are typically compiled to bytecode (class file) that can run on any Java Virtual Machine (JVM) regardless of computer architecture. Java is generalpurpose, concurrent, class-based, and object-oriented, and is specifically designed to have as few implementation dependencies as possible. It is intended to let application developers "write once, run anywhere". Java is considered by many as one of the most influential programming languages of the 20th century, and widely used from application software to web application.[9][10]

The original and reference implementation Java compilers, virtual machines, and class libraries were developed by Sun from 1995. As of May 2007, in compliance with the

specifications of the Java Community Process, Sun relicensed most of their Java technologies under the GNU General Public License. Others have also developed alternative implementations of these Sun technologies, such as the GNU Compiler for Java and GNU Classpath.

ADVANTAGES OF JAVA

1. **Java is simple:** Java was designed to be easy to use and is therefore easy to write, compile, debug, and learn than other programming languages. The reason that why Java is much simpler than C++ is because Java uses automatic memory allocation and garbage collection where else C++ requires the programmer to allocate memory and to collect garbage.
2. **Java is object-oriented:** Java is object-oriented because programming in Java is centered on creating objects, manipulating objects, and making objects work together. This allows you to create modular programs and reusable code.
3. **Java is platform-independent:** One of the most significant advantages of Java is its ability to move easily from one computer system to another.
4. **Java is distributed:** Distributed computing involves several computers on a network working together. Java is designed to make distributed computing easy with the networking capability that is inherently integrated into it.
5. **Java is interpreted:** An interpreter is needed in order to run Java programs. The programs are compiled into Java Virtual Machine code called bytecode.
6. **Java is secure:** Java is one of the first programming languages to consider security as part of its design. The Java language, compiler,

interpreter, and runtime environment were each developed with security in mind.

7. **Java is robust:** Robust means reliable and no programming language can really assure reliability. Java puts a lot of emphasis on early checking for possible errors, as Java compilers are able to detect many problems that would first show up during execution time in other languages.
8. **Java is multithreaded:** Multithreaded is the capability for a program to perform several tasks simultaneously within a program. In Java, multithreaded programming has been smoothly integrated into it, while in other languages, operating system-specific procedures have to be called in order to enable multithreading.

Operating System

□ Windows XP ,Windows 7 and further

HARDWARE & SOFTWARE REQUIREMENTS

Hardware Requirements:

Processor Brand	: Intel
Processor Type	: Core i3
Processor Speed	: 2 GHz

Processor Count : 1
RAM Size : 2 GB
Memory Technology : DDR₃
Computer Memory Type : DDR₃
SDRAM
Hard Drive Size : 160 GB

Software Requirements:

Operating system : Windows 11
Application server : JAVA (Swing & AWT)
Front end : JAVA
Connectivity : JDBC Driver
Database connectivity : WAMP (MYSQL Console).



To maintain the record of students, staffs, fees, and various users, in order to help the college administration.

REQUIREMENT SPECIFICATIONS

A Software Requirements Specification (SRS) is a complete description of the behavior of the system to be developed. It includes a set of use cases that describe all the interactions the users will have with the software. Use cases are also known as functional requirements. In addition to use cases, the SRS also contains non-functional (or supplementary) requirements. Non-functional requirements are requirements which impose constraints on the design or implementation (such as performance engineering requirements, quality standards, or design constraints).

The general requirements of the project are:

- **To maintain the admission details.**
- **To keep track the staff and students details.**
- **To record User details.**
- **To keep the record of various types of fee details.**
- **To keep the back-up of important data.**

Overview of Front End

An important issue for the development of a project is the selection of suitable front- end and back-end. When we decided to develop the project we went through an extensive study to determine the most suitable platform that suits the needs of the organization as well as helps in development of the project.

**The aspects of our study
included the following factors.**

Front-end selection:

1. It must have a graphical user interface that assists employees that are not from IT background.
2. Scalability and extensibility.
3. Flexibility.
4. Robustness.
5. According to the organization requirement and the culture.
6. Must provide excellent reporting features with good printing support.
7. Platform independent.
8. Easy to debug and maintain.
9. Event driven programming facility.

10. Front end must support some popular back end like MySQL.

About Java:

Java is a general-purpose, class-based, object-oriented programming language designed for having lesser implementation dependencies. It is a computing platform for application development. Java is fast, secure, and reliable, therefore. It is widely used for developing Java applications in laptops, data centers, game consoles, scientific supercomputers, cell phones, etc.

Overview of Back End Back

End Selection:

1. Multiple user support.
2. Efficient data handling.
3. Provide inherent features for security.
4. Efficient data retrieval and maintenance.
5. Stored procedures.
6. Popularity.
7. Operating System compatible.
8. Easy to install.
9. Various drivers must be available.

10. Easy to implant with the Front-end.

According to above stated features we selected MySQL as the backend.

The technical feasibility is frequently the most difficult area encountered at this stage. It is essential that the process of analysis and definition be conducted in parallel with an assessment to technical feasibility. It centers on the existing computer system (hardware, software etc.) and to what extent it can support the proposed system.

About SQL:

SQL is Structured Query Language, which is a computer language for storing, manipulating and retrieving data stored in a relational database.

SQL is the standard language for Relational Database System. All the Relational Database Management Systems (RDMS) like MySQL, MS Access, Oracle, Sybase, Informix, Postgres and SQL Server use SQL as their standard database language.

MySQL is a fast, easy-to-use RDBMS being used for many small and big businesses. MySQL is developed, marketed and supported by MySQL AB, which is a Swedish company. MySQL is becoming so popular because of many good reasons.[1]

MySQL is released under an open-source license. So you have nothing to pay to use it. MySQL is a very powerful program in its own right. It handles a large subset of the functionality of the most expensive and powerful database packages. MySQL uses a standard form of the well-known SQL data language. MySQL works on many operating systems and with many languages including PHP, PERL, C, C++, JAVA, etc.

MySQL works very quickly and works well even with large data sets. MySQL is very friendly to PHP, the most appreciated language for web development. MySQL supports large databases, up to 50 million rows or more in a table. The default file size limit for a table is 4GB, but you can increase this (if your operating system can handle it) to a theoretical limit of 8 million terabytes (TB).

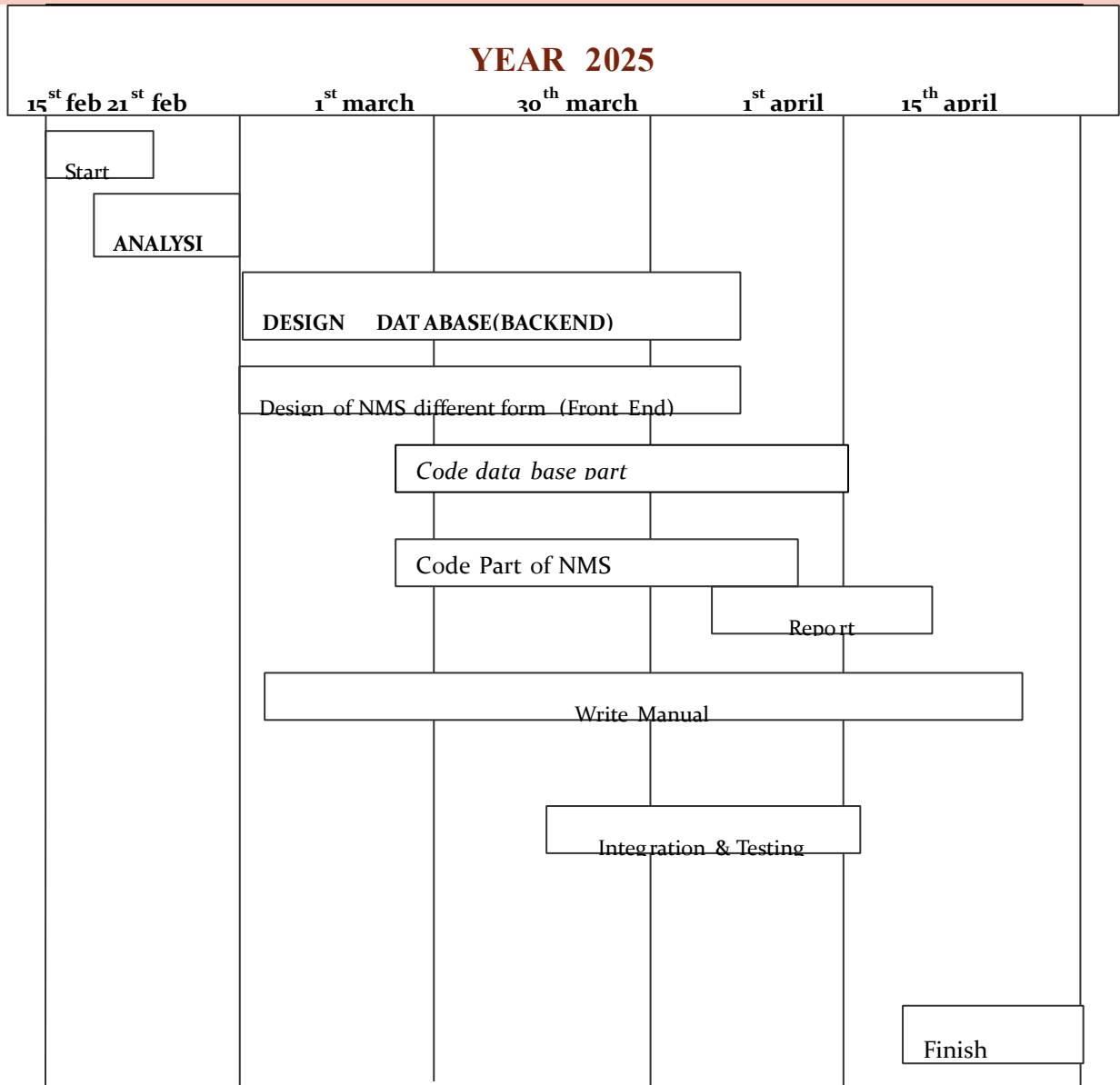
Also, they are using different dialects, such as –

- Oracle using PL/SQL,.
- SQL is widely popular because it offers the following advantages –
- Allows users to access data in the database management systems.
- Allows users to describe the data relational.
- Allows users to define the data in a database and manipulate that data.
- Allows to embed within other languages using SQL modules, libraries & pre-compilers.
- Allows users to create and drop databases and tables.
- Allows users to create view, stored procedure, functions in a database.
- Allows users to set permissions on tables, procedures and views.

GANTT CHART

Gantt charts (developed by Henry L. Gantt) are a project control technique that can be used for several purposes including scheduling, budgeting and resource planning. A Gantt chart is a bar chart with each bar representing an activity.

The bars are drawn against a time line. The length of each bar is proportional to the length of time planned for the activity. Gantt chart can take different phase depending on their intended use. The Gantt chart of **School Management System** is drawn for the time management. The Gantt chart is drawn below:

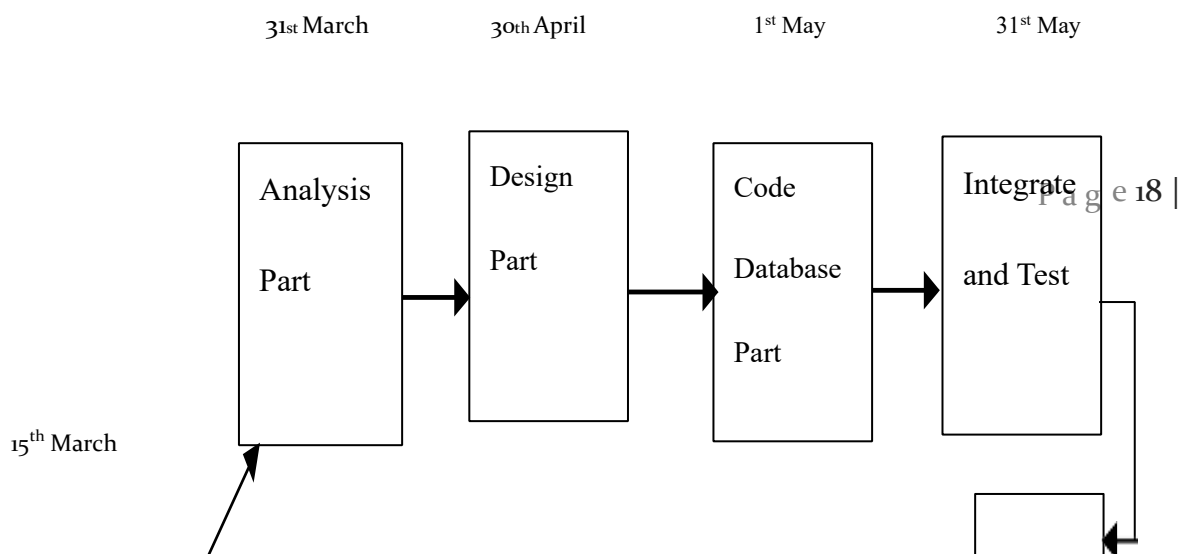


PERT CHART

PERT (Project Evaluation and Review Technique) charts consist of a network of boxes and arrows. The boxes represent activities and the arrows represent task dependencies.

PERT is organized by events and activities or tasks. PERT have more advantages and they are likely to be used for more complex projects. Through PERT chart the various task paths are defined. PERT enables the calculation of critical path. Each path consists of combination of tasks, which must be completed. The time and the cost associated with each task along a path are calculated, and the path that requires the greatest amount of elapsed time is the critical path. Calculation of the critical path enables project manager to monitor this series of task more closely than others to shift resources to it, if it begins to fall behind schedule.

PERT controls time and cost during the project and also facilitate finding the right balance between completing a project on time and completing it within the budget. There are thus not one but many critical paths, depending on the permutations of the estimates for each task. This makes analysis of critical path in PERT charts very complex. The PERT chart representation of the **School Management System** is shown below.



SYSTEM ANALYSIS

IDENTIFICATION OF NEED

To Important Indication for software we use the Principles of Requirement Engineering. Requirement engineering provides the appropriate mechanism for understanding what the customer wants, analyzing need, assessing feasibility, negotiating a reasonable solution, specifying the solution unambiguously, validating the specification and managing the requirement as they are transformed into an operational system. The requirement engineering process can be described in five distinct steps:

Requirement elicitation.

Requirement analysis & negotiation.

Requirement specification.

System Modeling.

Requirement validation.

Requirement Management.

In other words we can say that requirement analysis is a software task that bridges the gap between system level requirement engineering and software design.

Requirement analysis allows the software engineering to refine the software allocation and build models of the data, functional and behavioral domains that will be treated by software. Requirement analysis provides the software designer with a representation of information, function and behavior that can be translated into data, architectural, interface and component level design; finally the requirement specification provides the developer and the customer with the means to assess quality once software is built.

The most commonly used requirement elicitation technique is to conduct a meeting or interview. The first meeting between a software engineer and customer can be likened to the awareness of a first date between two adolescents. Neither person knows what to

say or ask; Both are worried that what they do say will be misinterpreted; both are thinking about where it might lead (Both likely have radically different expectations here); Both want to get the thing over with, but at the same time, both want it to be a success.

Here according to this principle the analyst starts by asking context-free-questions. That is a set of questions that will lead to a basic understanding of the problem, the people who want a solution, the nature of solution that is desired, and the effectiveness of the first encounter itself. The first set of Context-free questions focuses on the customer, the overall goals, and the benefits. For example, the analyst might ask: -

Who is behind the request for this Work?

Who will use the solution?

What will be economic benefit of a successful solution?

Is there another source for the solution that you need?

The next set of questions enables the software engineer to gain a better Understanding of the problem and the customer to voice his or her perceptions about a solution: -

How would you characterize “good” output that would be generated by a successful solution?

What problem(s) will this solution address?

Can you show me (or describe) the environment in which the solution will be used?

Will special performance issues or constraints affect the way the solution is approached?

The final set of question focuses on the effectiveness of the meeting:

Are you the right person to answer these questions? Are your answers?

Official”?

Are my questions relevant to the problem that you have?

Am I asking too many questions?

Can anyone else provide additional information?

Should I be asking you anything else?

According to the above concepts I went to institution and met its staffs, management, and some persons related with the organizational work, which advised me related with my project such as details of agent, policy related information will be cater this system.

Preliminary Investigation: -

The first step in the system development life cycle is the preliminary investigation to determine the feasibility of the System. The purpose of the preliminary investigation is to evaluate project requests. It is not a design study nor does it includes the collection of details top describe the business system in all respect. Rather, it is collecting of information that helps committee members to evaluate the merits of the project request and make an informed judgment about the feasibility of the proposed project.

Here for the Ngo Management System, I have worked on the preliminary investigation that accomplished the following objectives: -

Clarify and understand the project request.

Determine the size of the project.

Assess costs and benefits of alternative approaches.

Determine the technical and operational; feasibility of alternative approaches.

Report the findings to management, with recommendation outlining the acceptances or rejection of the proposal.

Considering above criteria I also keep in mind that the requirements are clearly understandable when the clarification of project request is enquired. The data of the Ngo Management System, which are collected by me during preliminary investigation, are through Reviewing organization Documents Onsite observation and

Conducting interviews.

FEASIBILITY STUDY

Not everything imaginable is feasible, not even in software, even as it may appear to outsiders. Feasibility is the determination of whether or not a project is worth doing. On the contrary, software feasibility has seven solid dimensions as below: -

- **Technical feasibility.**
- **Operational feasibility.**
- **Economic feasibility.**
- **Social feasibility.**
- **Management feasibility.**
- **Legal feasibility. □ Time feasibility.**

The process followed in making this determination is called a feasibility study. The type of study determines if a project can and should be taken. Once it has been determined that a project is feasible, that

analyst can go ahead and prepare the project specification which finalizes project requirements. Generally, feasibility studies are undertaken within tight time constraints and normally culminate in a written and oral feasibility report. The contents and recommendation of such a study will be used as a sound basis for deciding whether to proceed, postpone or cancel the project. Thus, since the feasibility study may lead to the commitment of large resources, it becomes necessary that it should be conducted competently and that no fundamental errors of judgment are made.

In other words we can say that a feasibility study is conducted to select the best system that meets performance requirement. This contains and identification description, an evaluation of candidate system and the selection of best system for the job. The system required a statement of constraints; the identification of specific system objective and a description of outputs define performance.

TECHNICAL FEASIBILITY: -

Technical feasibility is concerned with specifying equipment and software that will successfully satisfy the user requirements. My project is fit in technical feasibility because it requires neither too much heavy configuration system for its run nor in its development.

Following hardware and software components are used in the development of proposed system.

The same configuration system is also meets all the requirements to run or for implements to run or for implementation of the system.

Though I have also tested this software for that systems whose configuration is lower than the above specified

ECONOMICAL FEASIBILITY: -

Economic analysis is the most frequently used technique for evaluation of the effectiveness of a proposed system. More commonly known as Cost/Benefit analysis, the procedure is to determine the benefits and saving that are expected from a proposed system and compare them with costs. If benefits outweigh costs, a decision is taken to design and implement the system. Other weight costs, a decision is taken to design and implement the system. Otherwise, further justification or alternative in the proposed system will have to be made if is to have a chance of being approved. This is an ongoing effort that improves in accuracy at each phase of the system life cycle.

Before establishing Economical Feasibility so as to justify the expenses that different organization working under this category has to bear for this Software. It is crystal clear that the user or Customer of this Software has no need to invest heavy amount for this software. As my project has been written in such a language, which is a platform Independent. It has also the feature of WORM. It means it possess the technology of Write Once & Read Many.

There is no any heavy expense on the maintenance of this Software. Also there is no need to expense a heavy amount on the set of “Very Expert Personals” because it is very easy and user-friendly software. It is

fully tighten protection of Secret Password and User Name for unauthorized access of this Software.

For different types of Process or Operation, Reports or Query and Output also, there is no need of heavy expenditure.

OPERATIONAL FEASIBILITY: -

My project is also fit in Operational Feasibility Study. As I have explained before this about Technical Feasibility and Economical Feasibility, this Software is very easy and user-friendly.

I have used a very easy menu system and also used some control buttons in the easy way by which one can choose the options on his desire.

Though this is fully protected with the security by word (Password and user name), if both are matched only then one can get access of this software.

Various Reports, Forms and Queries can be generated on the fingertips for the user. In nutshell we can say that it has following operational features:

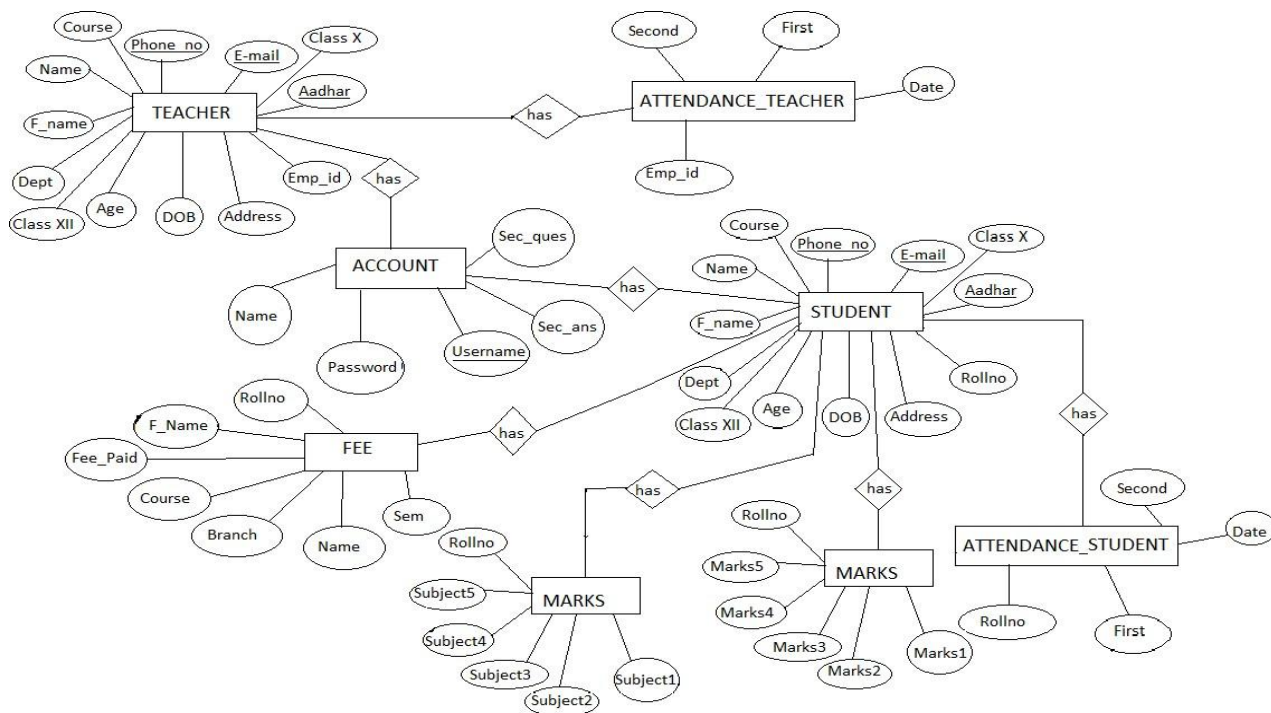
1. It is User-Friendly.
2. It is having less paperwork.
3. Efficient tractability.
4. Query can be generated.
5. Various Reports and Forms can be generated.
6. Fully protected by Password and User name for unauthorized access.

REQUIREMENT ANALYSIS

ER Diagram: ER Diagram is a high-level conceptual data model diagram. Entity-Relation model is based on the notion of real-world entities and the relationship between them. ER modelling helps you to analyse data requirements systematically to produce a well-designed database.

E-R DIAGRAM:

The DFD is an excellent communication tool for analysts to model processes and functional requirements. Used effectively, it is a useful and easy to understand modeling tool. It has broad application



SCHEMA DIAGRAM:

Schema diagram A schema diagram is the skeleton structure that represents the logical view of the entire database. It contains a descriptive detail of the database.

ACCOUNT:

<u>Username</u>	Name	Password	Sec_ques	Sec_ans
-----------------	------	----------	----------	---------

STUDENT:

Name	F_name	Age	DoB	Address	Class X	Branch	<u>Phoneno</u>	<u>E-mail</u>	Course	<u>Aadhar</u>	ClassXII	<u>Rollno</u>
------	--------	-----	-----	---------	---------	--------	----------------	---------------	--------	---------------	----------	---------------

TEACHER:

Name	F_name	Age	DoB	Address	Class X	Dept	<u>Phoneno</u>	<u>E-mail</u>	Course	<u>Aadhar</u>	ClassXII	<u>Emp_id</u>
------	--------	-----	-----	---------	---------	------	----------------	---------------	--------	---------------	----------	---------------

SUBJECT:

<u>Rollno</u>	Subject1	Subject2	Subject3	Subject4	Subject5
---------------	----------	----------	----------	----------	----------

MARKS:

<u>Rollno</u>	Marks1	Marks2	Marks3	Marks4	Marks5
---------------	--------	--------	--------	--------	--------

FEE:

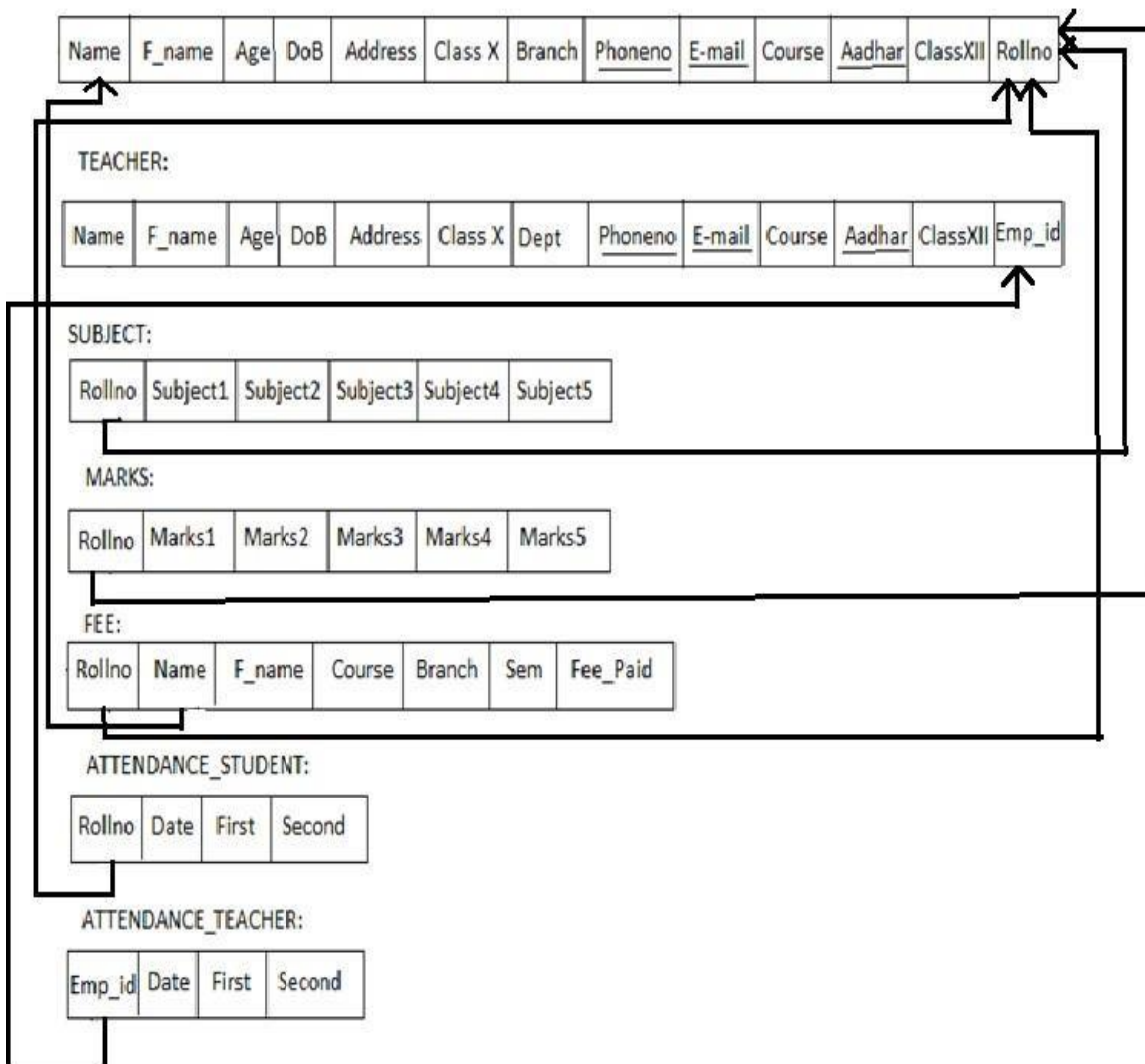
<u>Rollno</u>	<u>Name</u>	<u>F_name</u>	Course	Branch	Sem	<u>Fee_Paid</u>
---------------	-------------	---------------	--------	--------	-----	-----------------

ATTENDANCE_STUDENT:

<u>Rollno</u>	<u>Date</u>	<u>First</u>	<u>Second</u>
---------------	-------------	--------------	---------------

ATTENDANCE_TEACHER:

<u>Emp_id</u>	<u>Date</u>	<u>First</u>	<u>Second</u>
---------------	-------------	--------------	---------------



MODULE DESCRIPTION

These are the following Modules in the project:-

1. **STUDENT** :- This module is used for the information about the students through which it can be considered for particular student information as roll_no, address, course, admission status etc. In this module admission, student record updation , deletion etc happen.
2. **STAFF/EMPLOYEE** :- This module consists, the information about employees (working members) in the college for their different posts as faculties, coordinator, peon and other teaching & non-teaching staffs.
3. **FEE** : - This module maintains the information of student's admission fee , monthly fee and different type of fee Dues etc.
4. **SETTING** :- It is necessary to maintain the security level by implementing the concept of user authentication checking. The security module deals with user authentication and security measures. In which the user login id and password is used to secure & maintain the protection feature.
5. **REPORT** :-This module is used to view or print different type of Report like admission details, fee details etc.

TABLE DESCRIPTION

DATABASE Design

ACCOUNT TABLE

Account Table:

Account table: consists Of five attributes which are Username, Name, Password, Sec_ques, Sec_ans. Username is used as Primary key.

Desc account

```
mysql> desc account;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| username   | varchar(30)   | NO   | PRI | NULL    |       |
| name       | varchar(40)   | YES  |     | NULL    |       |
| password   | varchar(30)   | YES  |     | NULL    |       |
| sec_ques   | varchar(100)  | YES  |     | NULL    |       |
| sec_ans    | varchar(50)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Fig 5.1 Account table description

STUDENT TABLE

Student table :Student table is used to add the details of new student like Name,phoneno.,DoB,course,Branch etc...Phoneno. ,E-mail and Aadhar are used as Primary key.

Desc student;

```
mysql> desc student;
```

Field	Type	Null	Key	Default	Extra
name	varchar(20)	YES		NULL	
fathers_name	varchar(20)	YES		NULL	
age	varchar(5)	YES		NULL	
dob	varchar(20)	YES		NULL	
address	varchar(30)	YES		NULL	
phone	varchar(15)	NO	PRI	NULL	
email	varchar(25)	NO	PRI	NULL	
class_x	varchar(10)	YES		NULL	
class_xii	varchar(10)	YES		NULL	
aadhar	varchar(15)	NO	PRI	NULL	
rollno	varchar(15)	YES		NULL	
course	varchar(10)	YES		NULL	
branch	varchar(20)	YES		NULL	

13 rows in set (0.00 sec)

Student table description.

TEACHER TABLE:

Teacher table: Teacher table is used to add the details of new student like Name, phoneno.,DoB, course,Branch etc...Phoneno. ,E-mail and Aadhar are used as Primary key.

Desc teacher;

```
mysql> desc teacher;
```

Field	Type	Null	Key	Default	Extra
name	varchar(20)	YES		NULL	
fathers_name	varchar(20)	YES		NULL	
age	varchar(5)	YES		NULL	
dob	varchar(20)	YES		NULL	
address	varchar(30)	YES		NULL	
phone	varchar(15)	NO	PRI	NULL	
email	varchar(25)	NO	PRI	NULL	
class_x	varchar(10)	YES		NULL	
class_xii	varchar(10)	YES		NULL	
aadhar	varchar(15)	NO	PRI	NULL	
course	varchar(10)	YES		NULL	
emp_id	varchar(15)	YES		NULL	
dept	varchar(20)	YES		NULL	

```
13 rows in set (0.00 sec)
```

ATTENDANCE_STUDENT TABLE

Attendance_Student Table: Attendance_Student table is used to mark the attendance of the student day to day which as attributes like rollno,name,first and second half.

Desc attendance_student;

```
mysql> desc attendance_student;
```

Field	Type	Null	Key	Default	Extra
rollno	varchar(20)	YES		NULL	
Date	varchar(30)	YES		NULL	
first	varchar(10)	YES		NULL	
second	varchar(10)	YES		NULL	

4 rows in set (0.04 sec)

Attendance_Student table description.

ATTENDANCE_TEACHER TABLE

Attendance_Teachertable Attendance_Teacher table is used to mark the attendance of the teacher day to day which as attributes like emp_id,name,first and second half.

Desc attendance_teacher;

```
mysql> desc attendance_teacher;
```

Field	Type	Null	Key	Default	Extra
emp_id	varchar(20)	YES		NULL	
Date	varchar(30)	YES		NULL	
first	varchar(10)	YES		NULL	
second	varchar(10)	YES		NULL	

4 rows in set (0.00 sec)

Attendance_Teacher table description.

SUBJECT TABLE

Subject table :Subject table is used to add the subjects of the student in that particular sem with the attributes like rollno and five subjects.

Desc Subject;

```
mysql> desc subject;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| rollno     | varchar(25)   | YES  |     | NULL    |       |
| subject1   | varchar(30)   | YES  |     | NULL    |       |
| subject2   | varchar(30)   | YES  |     | NULL    |       |
| subject3   | varchar(30)   | YES  |     | NULL    |       |
| subject4   | varchar(30)   | YES  |     | NULL    |       |
| subject5   | varchar(30)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.02 sec)
```

Subject table description.

MARKS TABLE

Marks table :Marks table is used to add the marks of the particular subjects of the student in a particular sem and the attributes used are rollno and five subject marks.

Desc Marks;

```
mysql> desc marks;
```

Field	Type	Null	Key	Default	Extra
rollno	varchar(15)	YES		NULL	
marks1	varchar(20)	YES		NULL	
marks2	varchar(20)	YES		NULL	
marks3	varchar(20)	YES		NULL	
marks4	varchar(20)	YES		NULL	
marks5	varchar(20)	YES		NULL	

6 rows in set (0.03 sec)

Marks table description.

FEE TABLE

Fee table: fee table is used to pay the fee dues of the student for that particular sem and the attributes used like rollno, name, fathersname, course, branch, sem and fee_paid.

Desc Fee;

```
mysql> desc fee;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| rollno         | varchar(20)   | YES  |     | NULL    |       |
| name           | varchar(25)   | YES  |     | NULL    |       |
| fathers_name   | varchar(25)   | YES  |     | NULL    |       |
| course         | varchar(10)   | YES  |     | NULL    |       |
| branch         | varchar(20)   | YES  |     | NULL    |       |
| semester       | varchar(10)   | YES  |     | NULL    |       |
| fee_paid       | varchar(15)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.02 sec)
```

Fee table description.

TABLE WITH VALUES

Output design:

Account table :Account table consists of five attributes which will be retrived from user when the user signup/logs in.

Select * from account;

Account table

```
mysql> select * from account;
```

username	name	password	sec_ques	sec_ans
raja	RAJA	12345	Your Lucky Number?	9900
gopi	Gopi	gopi123	Your NickName?	gopi
vikas	VIKAS	sai12	Your child SuperHero?	ntr
mohan	MOHAN	mogan	Your childhood Name ?	mogan
akash	AKASH	67890	Your Lucky Number?	9

```
5 rows in set (0.00 sec)
```

Student table: Student table is used to add the details of new student like

Name,phoneno.,DoB,course,Branch etc...Phoneno.E-mail and Aadhar are used as Primary key.

Select * from student;

Student table

```
mysql> select * from student;
```

name	fathers_name	age	dob	address	phone	email	class_x	class_xii	aadhar	rollno	course	branch
Vikas	Sai	22	02/03/1998	Bangalore	9869869576	vikasvicky11@gmail.com	84	77	229876589745	15331807	M.Tech	Electronics
Raja	Srinu	21	29/05/1999	Bangalore	9897969984	raja123@gmail.com	88	82	676476486745	15335115	M.Tech	Mechanical
Gopi	Krishna	20	03/10/2000	Kolar	7869687696	gopi11@gmail.com	82	78	885787588758	1533842	B.Tech	Computer Science
Akash	Kumar	20	22/08/2000	Mangalore	7879696896	akash1122@gmail.com	84	81	906895709687	15339828	B.Tech	Civil
Mohan	Mogesh	19	18/02/2001	Bangalore	7869869665	mogan11@gmail.com	82	79	987689786988	15333481	B.Com	Professional Degree

5 rows in set (0.00 sec)

Teacher table :Teacher table is used to add the details of new student like

Name,phoneno.,DoB,course,Branch etc...Phoneno.,E-mail and Aadhar are used as Primary key.

Select * from teacher;

Teacher table

```
mysql> select * from teacher;
```

name	fathers_name	age	dob	address	phone	email	class_x	class_xii	aadhar	course	emp_id	dept
Lakshmi	Venkatesh	45	04/05/1975	Bangalore	7897658656	lakshmi12@gmail.com	83	78	756876487594	Msc	1016569	Computer Science
Prakash	Kumarswamy	54	21/03/1966	Bangalore	9867976976	prakash11@gmail.com	84	81	979477658798	M.Tech	1013079	Mechanical
Naveen.B.M	Bhaskar	38	26/11/1982	Bangalore	8978987687	naveen123@gmail.com	87	77	896596796798	MBA	1012340	Others
Mahesh.G	Ganesh	41	16/09/1979	Mangalore	7897869876	maheshg11@gmail.com	78	68	456736753857	MCA	1014233	Others
Rakesh	Chandrasekhar	36	11/06/1984	Mysore	8876659766	rakesh121@gmail.com	88	87	337659876007	BCom	1012307	Professional Degree

5 rows in set (0.00 sec)

Attendance_Student table :Attendance_Student table is used to mark the attendance of the student day to day which as attributes like rollno,name,first and second half.

Select * from attendance_student;

Attendance_student table

```
mysql> select * from attendance_student;
```

rollno	Date	first	second
15331807	Thu Jan 14 16:12:03 IST 2021	Present	Present
15335115	Thu Jan 14 16:12:15 IST 2021	Present	Absent
1533842	Thu Jan 14 16:12:27 IST 2021	Absent	Present
15339828	Thu Jan 14 16:12:41 IST 2021	Absent	Absent
15333481	Thu Jan 14 16:13:00 IST 2021	Leave	Leave

5 rows in set (0.00 sec)

Attendance_Teacher table

:Attendance_Teachertable is used to mark the attendance of the teacher day to day which as attributes like emp_id,name,first and second half.

Select * from attendance_teacher;

Attendance_teacher table

```
mysql> select * from attendance_teacher;
```

emp_id	Date	first	second
1016569	Thu Jan 14 15:45:45 IST 2021	Present	Present
1013079	Thu Jan 14 15:46:00 IST 2021	Absent	Present
1012340	Thu Jan 14 15:46:15 IST 2021	Present	Absent
1014233	Thu Jan 14 15:46:32 IST 2021	Absent	Absent
1012307	Thu Jan 14 15:46:47 IST 2021	Leave	Leave

5 rows in set (0.00 sec)

Subject table : Subject table is used to add the subjects of the student in that particular sem with the attributes like rollno and five subjects.

Select * from Subject;

```
mysql> select * from subject;
```

rollno	subject1	subject2	subject3	subject4	subject5
15331807	Devices	Signals	System	Numericals	Circuits
15335115	Mathematics	Statics and Dynamics	Solid mechanics	Material engineering	Composites
1533842	Computer networks	Database management	Python	Unix	ATC
15339828	Building materials	Strength of materials	Structures	Contuction project	Steel design
15333481	Accounts	Economics	Statistics	Management	Finance

5 rows in set (0.00 sec)

Subject table

Select *
from Marks;

Fee table :fee table is used to pay the fee
dues of the student for that

particularM sem and the attributes
used a

like^rk rollno, name, fathersname,
course, s

branch, sem t and
fee_paid. a

b

Select * from Fee;

```
mysql> select * from marks;
```

rollno	marks1	marks2	marks3	marks4	marks5
15331807	78	82	79	76	85
15335115	78	83	88	79	80
1533842	77	68	76	68	70
15339828	60	68	65	73	75
15333481	78	72	70	69	74

```
5 rows in set (0.00 sec)
```

IMPLEMENTATION

SPLASH CODE

```
Package university.management.system;
```

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
public class Splash extends JFrame implements Runnable {
```

```
    Thread t;
```

```
    Splash(){
```

```
        ImageIcon i1 = new ImageIcon(ClassLoader.getResource("icon/first.png"));
```

```
        Image i2 = i1.getImage().getScaledInstance(1000,700, Image.SCALE_DEFAULT);
```

```
        ImageIcon i3 = new ImageIcon(i2);
```

```
        JLabel img = new JLabel(i3);
```

```
        add(img);
```

```
        t = new Thread(this);
```

```
        t.start();
```

```
        setVisible(false);
```

```
        int x = 1;
```

```
        for (int i =2; i<=600; i+=4, x+=1){
```

```
            setLocation(600 - ((i + x)/2), 350 - (i/2));
```

```
            setSize(i+3 *x, i+x/2);
```

```
            try{
```

```
                Thread.sleep(10);
```

```
            }catch (Exception e){
```

```
                e.printStackTrace();
```

```
            }
```

```

    }

}

public void run(){
    try{
        Thread.sleep(7000);
        setVisible(false);
        new Login();

    }catch(Exception e){
        e.printStackTrace();
    }
}

public static void main(String[] args) {
    new Splash();
}
}

```

Sample code for ABOUT

```

package university.management.system;

import javax.swing.*.*;
import java.awt.*.*;

public class About extends JFrame {
    About(){

        ImageIcon i1 = new
        ImageIcon(ClassLoader.getResource("icon/college.jpg"));

```



```
Image i2 = i1.getImage().getScaledInstance(300,200,Image.SCALE_DEFAULT);
ImageIcon i3 = new ImageIcon(i2);
JLabel img = new JLabel(i3);
img.setBounds(350,0,300,200);
add(img);
```

```
JLabel heading = new JLabel("<html>CIMAGE COLLEGE </html>");
heading.setBounds(70,20,300,130);
heading.setFont(new Font("Tahoma",Font.BOLD,30));
add(heading);
```

```
JLabel name = new JLabel("Cimage college");
name.setBounds(60,260,550,40);
name.setFont(new Font("Tahoma",Font.BOLD,30));
add(name);
```

```
JLabel contact = new JLabel("cimagecollege@gmail.com");
contact.setBounds(70,340,550,40);
contact.setFont(new Font("Tahoma",Font.BOLD,30));
add(contact);
```

```
setSize(700,500);
setLocation(400,150);
getContentPane().setBackground(new Color(252,228,210));
setLayout(null);
setVisible(true);
```

```
}
```

```
public static void main(String[] args) {
    new About();
```

```
}  
}
```

Sample code for CONN

```
package university.management.system;
```

```
import java.rmi.ConnectException;  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.Statement;
```

```
public class Conn {
```

```
    Connection connection;  
    Statement statement;
```

```
    Conn(){
```

```
        try{
```

```
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```
            connection =
```

```
DriverManager.getConnection("jdbc:mysql://localhost:3306/universitymanagement"  
        ,"root","Ashish@777");
```

```
            statement = connection.createStatement();
```

```
        }catch (Exception e){
```

```
            e.printStackTrace();
```

```
        }
```

```
    }
```

```
}
```

LOGIN PAGE

```
package university.management.system;

import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.Connection;
import java.sql.ResultSet;

public class Login extends JFrame implements ActionListener {

    JTextField textFieldName;

    JPasswordField passswordfield;
    JButton login, back;

    Login(){

        JLabel labelName = new JLabel("Username");
        labelName.setBounds(40,20,100,20);
        add(labelName);

        textFieldName = new JTextField();
        textFieldName.setBounds(150,20,150,20);
        add(textFieldName);

        JLabel labelpass = new JLabel("Password");
        labelpass.setBounds(40, 70,100,20);
        add(labelpass);
```

```
passswordfield = new JPasswordField();  
passswordfield.setBounds(150,70,150,20);  
add(passswordfield);
```

```
login = new JButton("Login");  
login.setBounds(40,140,120,30);  
login.setBackground(Color.BLACK);  
login.setForeground(Color.white);  
login.addActionListener(this);  
add(login);
```

```
back = new JButton("Back");  
back.setBounds(180,140,120,30);  
back.setBackground(Color.BLACK);  
back.setForeground(Color.white);  
back.addActionListener(this);  
add(back);
```

```
ImageIcon i1 = new  
ImageIcon(ClassLoader.getResource("icon/second.png"));  
Image i2 = i1.getImage().getScaledInstance(200,200,Image.SCALE_DEFAULT);  
ImageIcon i3 = new ImageIcon(i2);  
JLabel img = new JLabel(i3);  
img.setBounds(350,20,200,200);  
add(img);
```

```
ImageIcon i11 = new  
ImageIcon(ClassLoader.getResource("icon/loginback.png"));  
Image i22 = i11.getImage().getScaledInstance(600,300,Image.SCALE_DEFAULT);  
ImageIcon i33 = new ImageIcon(i22);  
JLabel image = new JLabel(i33);  
image.setBounds(0,0,600,300);
```

```
add(image);
```

```
setSize(600,300);  
setVisible(true);  
setLocation(500,250);  
setLayout(null);  
}
```

```
@Override
```

```
public void actionPerformed(ActionEvent e) {
```

```
    if (e.getSource() == login){
```

```
        String username = textFieldName.getText();
```

```
        String password = passswordfield.getText();
```

```
        String query = "select * from login where username='"+username+"' and  
password='"+password+"'";
```

```
        try{
```

```
            Conn c = new Conn();
```

```
            ResultSet resultSet = c.statement.executeQuery(query);
```

```
            if (resultSet.next()){
```

```
                setVisible(false);
```

```
                new main_class();
```

```
            }else {
```

```
                JOptionPane.showMessageDialog(null,"Invalid username or password");
```

```
        }
```

```
    }catch (Exception E){
```

```
        E.printStackTrace();
```

```
    }
```

```

        }else{
            setVisible(false);
        }

    }

    public static void main(String[] args) {
        new Login();

    }
}

```

MAIN CLASS

```

package university.management.system;

import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class main_class extends JFrame implements ActionListener {
    main_class(){
        ImageIcon i1 = new
        ImageIcon(ClassLoader.getResource("icon/university.jpg"));
        Image i2 = i1.getImage().getScaledInstance(1540,750,Image.SCALE_DEFAULT);
        ImageIcon i3 = new ImageIcon(i2);
        JLabel img = new JLabel(i3);
        add(img);
    }
}

```

```
JMenuBar mb = new JMenuBar();
```

```
JMenu newInfo = new JMenu("New Information");  
newInfo.setForeground(Color.BLACK);  
mb.add(newInfo);
```

```
JMenuItem facultyInfo = new JMenuItem("New Faculty Information");  
facultyInfo.setBackground(Color.WHITE);  
facultyInfo.addActionListener(this);  
newInfo.add(facultyInfo);
```

```
JMenuItem studentInfo = new JMenuItem("New Student Information");  
studentInfo.setBackground(Color.WHITE);  
studentInfo.addActionListener(this);  
newInfo.add(studentInfo);  
JMenu details = new JMenu("View Details");  
details.setForeground(Color.BLACK);  
details.addActionListener(this);  
mb.add(details);
```

```
JMenuItem facultydetails = new JMenuItem("View Faculty Details");  
facultydetails.setBackground(Color.WHITE);  
facultydetails.addActionListener(this);  
details.add(facultydetails);
```

```
JMenuItem studentdetails = new JMenuItem("View Student Details");  
studentdetails.setBackground(Color.WHITE);  
studentdetails.addActionListener(this);  
details.add(studentdetails);
```

```
JMenu leave = new JMenu("Apply Leave");  
leave.setForeground(Color.BLACK);  
leave.addActionListener(this);  
mb.add(leave);
```

```
JMenuItem facultyLeave = new JMenuItem(" Faculty Leave");  
facultyLeave.setBackground(Color.WHITE);  
facultyLeave.addActionListener(this);  
leave.add(facultyLeave);
```

```
JMenuItem studentLeave = new JMenuItem(" Student Leave");  
studentLeave.setBackground(Color.WHITE);  
studentLeave.addActionListener(this);  
leave.add(studentLeave);
```

```
JMenu leaveDetails = new JMenu("Leave Details");  
leaveDetails.setForeground(Color.BLACK);  
leaveDetails.addActionListener(this);  
mb.add(leaveDetails);
```

```
JMenuItem facultyLeaveDetails = new JMenuItem(" Faculty Leave Details");  
facultyLeaveDetails.setBackground(Color.WHITE);  
facultyLeaveDetails.addActionListener(this);  
leaveDetails.add(facultyLeaveDetails);
```

```
JMenuItem studentLeaveDetails = new JMenuItem(" Student Leave Details");  
studentLeaveDetails.setBackground(Color.WHITE);  
studentLeaveDetails.addActionListener(this);  
leaveDetails.add(studentLeaveDetails);
```



```
JMenu exam = new JMenu("Examination");  
exam.setForeground(Color.BLACK);  
exam.addActionListener(this);  
mb.add(exam);
```

```
JMenuItem ExaminationDetails = new JMenuItem("Examination Results");  
ExaminationDetails.setBackground(Color.WHITE);  
ExaminationDetails.addActionListener(this);  
exam.add(ExaminationDetails);
```

```
JMenuItem EnterMarks = new JMenuItem("Enter Marks");  
EnterMarks.setBackground(Color.WHITE);  
EnterMarks.addActionListener(this);  
exam.add(EnterMarks);
```

```
JMenu updateInfo = new JMenu("Update Details");  
updateInfo.setForeground(Color.BLACK);  
updateInfo.addActionListener(this);  
mb.add(updateInfo);
```

```
JMenuItem updatefacultyinfo = new JMenuItem("Update Faculty Details");  
updatefacultyinfo.setBackground(Color.WHITE);  
updatefacultyinfo.addActionListener(this);  
updateInfo.add(updatefacultyinfo);
```

```
JMenuItem updatestudentinfo = new JMenuItem("Update Student Details");  
updatestudentinfo.setBackground(Color.WHITE);  
updatestudentinfo.addActionListener(this);
```

```
updateInfo.add(updateStudentInfo);
```

```
JMenu fee = new JMenu("Fee Details");  
fee.setForeground(Color.BLACK);  
fee.addActionListener(this);  
mb.add(fee);
```

```
JMenuItem feeStructure = new JMenuItem("Fee Structure");  
feeStructure.setBackground(Color.WHITE);  
feeStructure.addActionListener(this);  
fee.add(feeStructure);
```

```
JMenuItem feeForm = new JMenuItem("Student Fee Form");  
feeForm.setBackground(Color.WHITE);  
feeForm.addActionListener(this);  
fee.add(feeForm);
```

```
JMenu utility = new JMenu("Utility");  
utility.setForeground(Color.BLACK);  
utility.addActionListener(this);  
mb.add(utility);
```

```
JMenuItem ca = new JMenuItem("Calculator");  
ca.setBackground(Color.WHITE);  
ca.addActionListener(this);  
utility.add(ca);
```

```
JMenuItem Notepad = new JMenuItem("Notepad");  
Notepad.setBackground(Color.WHITE);  
Notepad.addActionListener(this);  
utility.add(Notepad);
```

```
JMenu about = new JMenu("About");  
about.setForeground(Color.BLACK);  
about.addActionListener(this);  
mb.add(about);
```

```
JMenuItem About = new JMenuItem("About");  
About.setBackground(Color.WHITE);  
About.addActionListener(this);  
about.add(About);
```

```
JMenu exit = new JMenu("Exit");  
exit.setForeground(Color.BLACK);  
exit.addActionListener(this);  
mb.add(exit);
```

```
JMenuItem Exit = new JMenuItem("Exit");  
Exit.setBackground(Color.WHITE);  
Exit.addActionListener(this);  
exit.add(Exit);
```

```
setJMenuBar(mb);
```

```
setSize(1540,850);
```

```
    setVisible(true);  
}
```

```
@Override
```

```
public void actionPerformed(ActionEvent e) {  
    String sm = e.getActionCommand();  
    if (sm.equals("Exit")){  
        System.exit(15);  
    } else if (sm.equals("Calculator")) {  
        try{  
            Runtime.getRuntime().exec("calc.exe");  
        }catch (Exception E){  
            E.printStackTrace();  
        }  
  
    } else if (sm.equals("Notepad")) {  
        try {  
            Runtime.getRuntime().exec("notepad.exe");  
        }catch (Exception E){  
            E.printStackTrace();  
        }  
  
    } else if (sm.equals("New Faculty Information")) {  
        new AddFaculty();  
    } else if (sm.equals("New student Information")) {  
        new AddStudent();  
    } else if (sm.equals("View Faculty Details")) {  
        new TeacherDetails();  
    } else if (sm.equals("View Student Details")) {  
        new StudentDetails();  
    } else if (sm.equals("Faculty Leave")) {  
        new TeacherLeave();  
    }  
}
```

```

    } else if (sm.equals("Student Leave")) {
        new StudentLeave();
    } else if (sm.equals("Faculty Leave Details")) {
        new TeacherLeaveDetails();
    } else if (sm.equals("Student Leave Details")) {
        new StudentLeaveDetails();
    } else if (sm.equals("Update Faculty Details")) {
        new UpdateTeacher();
    } else if (sm.equals("Update Student Details ")) {
        new UpdateStudent();
    } else if (sm.equals("Enter Marks")) {
        new EnterMarks();
    } else if (sm.equals("Examination Result")) {
        new ExaminationDetails();
    } else if (sm.equals("Fee Structure")) {
        new FeeStructure();
    } else if (sm.equals("Student Fee Form")) {
        new StudentFeeForm();
    } else if (sm.equals("About")) {
        new About();
    }
}

public static void main(String[] args) {
    new main_class();
}
}

```

Sample code for Add Student

```
package university.management.system;

import com.toedter.calendar.JDateChooser;

import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.Random;

public class AddStudent extends JFrame implements ActionListener {
    JTextField textName, textfather, textAddress, textPhone, textemail,
    textM10, textM12, textAadhar;
    JLabel empText;
    JDateChooser cdob;
    JComboBox courseBox, departmentBox;
    JButton submit, cancel;

    Random ran = new Random();
    long f4 = Math.abs((ran.nextLong() % 9000L) + 1000L);

    AddStudent(){

        getContentPane().setBackground(new Color(128,176,255));

        JLabel heading = new JLabel("New Teacher Details");
        heading.setBounds(310,30,500,50);
        heading.setFont(new Font("serif",Font.BOLD,30));
        add(heading);

        JLabel name = new JLabel("Name");
```

```
name.setBounds(50,150,100,30);  
name.setFont(new Font("Serif",Font.BOLD,20));  
add(name);
```

```
textName = new JTextField();  
textName.setBounds(200,150,150,30);  
add(textName);
```

```
JLabel fname = new JLabel("Father Name");  
fname.setBounds(400,150,200,30);  
fname.setFont(new Font("Serif",Font.BOLD,20));  
add(fname);
```

```
textfather = new JTextField();  
textfather.setBounds(600,150,150,30);  
add(textfather);
```

```
JLabel roll = new JLabel("Roll Number");  
roll.setBounds(50,200,200,30);  
roll.setFont(new Font("Serif",Font.BOLD,20));  
add(roll);
```

```
empText = new JLabel("1409"+f4);  
empText.setBounds(200,200,150,30);  
empText.setFont(new Font("Serif",Font.BOLD,20));  
add(empText);
```

```
JLabel dob = new JLabel("Date Of Birth");  
dob.setBounds(400,200,200,30);
```

```
dob.setFont(new Font("Serif",Font.BOLD,20));  
add(dob);
```

```
cdob = new JDateChooser();  
cdob.setBounds(600,200,150,30);  
add(cdob);
```

```
JLabel address = new JLabel("Address");  
address.setBounds(50,250,200,30);  
address.setFont(new Font("Serif",Font.BOLD,20));  
add(address);
```

```
textAddress = new JTextField();  
textAddress.setBounds(200,250,150,30);  
add(textAddress);
```

```
JLabel phone = new JLabel("Phone");  
phone.setBounds(400,250,200,30);  
phone.setFont(new Font("Serif",Font.BOLD,20));  
add(phone);
```

```
textPhone = new JTextField();  
textPhone.setBounds(600,250,150,30);  
add(textPhone);
```

```
JLabel email = new JLabel("Email");  
email.setBounds(50,300,200,30);  
email.setFont(new Font("Serif",Font.BOLD,20));  
add(email);
```



```
textemail = new JTextField();  
textemail.setBounds(200,300,150,30);  
add(textemail);
```

```
JLabel M10 = new JLabel("Class X (%)");  
M10.setBounds(400,300,200,30);  
M10.setFont(new Font("Serif",Font.BOLD,20));  
add(M10);
```

```
textM10 = new JTextField();  
textM10.setBounds(600,300,150,30);  
add(textM10);
```

```
JLabel M12 = new JLabel("Class XII (%)");  
M12.setBounds(50,350,200,30);  
M12.setFont(new Font("Serif",Font.BOLD,20));  
add(M12);
```

```
textM12 = new JTextField();  
textM12.setBounds(200,350,150,30);  
add(textM12);
```

```
JLabel AadharNo = new JLabel("Aadhar Number");  
AadharNo.setBounds(400,350,200,30);  
AadharNo.setFont(new Font("Serif",Font.BOLD,20));  
add(AadharNo);
```

```
textAadhar = new JTextField();  
textAadhar.setBounds(600,350,150,30);  
add(textAadhar);
```

```
JLabel Qualification = new JLabel("Course");  
Qualification.setBounds(50,400,200,30);  
Qualification.setFont(new Font("Serif",Font.BOLD,20));  
add(Qualification);
```

```
String course[] = {"B.Tech", "BBA", "BCA", "BSC","MCA","MBA","B.COM"};  
courseBox = new JComboBox(course);  
courseBox.setBounds(200,400,150,30);  
courseBox.setBackground(Color.WHITE);  
add(courseBox);
```

```
JLabel Department = new JLabel("Branch");  
Department.setBounds(400,400,200,30);  
Department.setFont(new Font("Serif",Font.BOLD,20));  
add(Department);
```

```
String department[] = {"Computer  
Science", "Electronics", "Mechanical", "Civil", "IT"};  
departmentBox = new JComboBox(department);  
departmentBox.setBounds(600,400,150,30);  
departmentBox.setBackground(Color.WHITE);  
add(departmentBox);
```

```
submit = new JButton("Submit");  
submit.setBounds(250,550,120,30);  
submit.setBackground(Color.BLACK);  
submit.setForeground(Color.WHITE);  
submit.addActionListener(this);  
add(submit);
```

```
cancel = new JButton("Cancel");
cancel.setBounds(450,550,120,30);
cancel.setBackground(Color.BLACK);
cancel.setForeground(Color.WHITE);
cancel.addActionListener(this);
add(cancel);
```

```
setSize(900,700);
setLocation(350,50);
setLayout(null);
setVisible(true);
```

```
}
```

@Override

```
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == submit){
        String name = textName.getText();
        String fname = textfather.getText();
        String empid = empText.getText();
        String dob = ((JTextField) cdob.getDateEditor().getUiComponent()).getText();
        String address = textAddress.getText();
        String phone = textPhone.getText();
        String email = textemail.getText();
        String x = textM10.getText();
        String xii = textM12.getText();
        String aadhar = textAadhar.getText();
        String course = (String) courseBox.getSelectedItem();
        String department = (String) departmentBox.getSelectedItem();
        try{
```

```

        String q = "insert into student
values('"+name+"','"+fname+"','"+empid+"','"+dob+"','"+address+"','"+phone+"','"+e
mail+"','"+x+"','"+xii+"','"+aadhar+"','"+course+"','"+department+"')";
        Conn c = new Conn();
        c.statement.executeUpdate(q);
        JOptionPane.showMessageDialog(null,"Details Inserted");
        setVisible(false);

    }catch (Exception E){
        E.printStackTrace();
    }
    }else{
        setVisible(false);
    }
}

public static void main(String[] args) {
    new AddStudent();
}
}

```

Add Faculty

```

package university.management.system;

import com.toedter.calendar.JDateChooser;

```

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.Random;
```

```
public class AddFaculty extends JFrame implements ActionListener {
```

```
    JTextField textName, textfather, textAddress, textPhone, textemail,
    textM10, textM12, textAadhar;
```

```
    JLabel empText;
```

```
    JDateChooser cdob;
```

```
    JComboBox courseBox, departmentBox;
```

```
    JButton submit, cancel;
```

```
    Random ran = new Random();
```

```
    long f4 = Math.abs((ran.nextLong() % 9000L) + 1000L);
```

```
    AddFaculty(){
```

```
        getContentPane().setBackground(new Color(166,164,252));
```

```
        JLabel heading = new JLabel("New Teacher Details");
```

```
        heading.setBounds(310,30,500,50);
```

```
        heading.setFont(new Font("serif",Font.BOLD,30));
```

```
        add(heading);
```

```
        JLabel name = new JLabel("Name");
```

```
name.setBounds(50,150,100,30);  
name.setFont(new Font("Serif",Font.BOLD,20));  
add(name);
```

```
textName = new JTextField();  
textName.setBounds(200,150,150,30);  
add(textName);
```

```
JLabel fname = new JLabel("Father Name");  
fname.setBounds(400,150,200,30);  
fname.setFont(new Font("Serif",Font.BOLD,20));  
add(fname);
```

```
textfather = new JTextField();  
textfather.setBounds(600,150,150,30);  
add(textfather);
```

```
JLabel empID = new JLabel("Employee ID");  
empID.setBounds(50,200,200,30);  
empID.setFont(new Font("Serif",Font.BOLD,20));  
add(empID);
```

```
empText = new JLabel("1409"+f4);  
empText.setBounds(200,200,150,30);  
empText.setFont(new Font("Serif",Font.BOLD,20));  
add(empText);
```

```
JLabel dob = new JLabel("Date Of Birth");  
dob.setBounds(400,200,200,30);  
dob.setFont(new Font("Serif",Font.BOLD,20));  
add(dob);
```

```
cdob = new JDateChooser();  
cdob.setBounds(600,200,150,30);  
add(cdob);
```

```
JLabel address = new JLabel("Address");  
address.setBounds(50,250,200,30);  
address.setFont(new Font("Serif",Font.BOLD,20));  
add(address);
```

```
textAddress = new JTextField();  
textAddress.setBounds(200,250,150,30);  
add(textAddress);
```

```
JLabel phone = new JLabel("Phone");  
phone.setBounds(400,250,200,30);  
phone.setFont(new Font("Serif",Font.BOLD,20));  
add(phone);
```

```
textPhone = new JTextField();
```

```
textPhone.setBounds(600,250,150,30);  
add(textPhone);
```

```
JLabel email = new JLabel("Email");  
email.setBounds(50,300,200,30);  
email.setFont(new Font("Serif",Font.BOLD,20));  
add(email);
```

```
textemail = new JTextField();  
textemail.setBounds(200,300,150,30);  
add(textemail);
```

```
JLabel M10 = new JLabel("Class X (%)");  
M10.setBounds(400,300,200,30);  
M10.setFont(new Font("Serif",Font.BOLD,20));  
add(M10);
```

```
textM10 = new JTextField();  
textM10.setBounds(600,300,150,30);  
add(textM10);
```

```
JLabel M12 = new JLabel("Class XII (%)");  
M12.setBounds(50,350,200,30);  
M12.setFont(new Font("Serif",Font.BOLD,20));  
add(M12);
```



```
textM12 = new JTextField();  
textM12.setBounds(200,350,150,30);  
add(textM12);
```

```
JLabel AadharNo = new JLabel("Aadhar Number");  
AadharNo.setBounds(400,350,200,30);  
AadharNo.setFont(new Font("Serif",Font.BOLD,20));  
add(AadharNo);
```

```
textAadhar = new JTextField();  
textAadhar.setBounds(600,350,150,30);  
add(textAadhar);
```

```
JLabel Qualification = new JLabel("Qualification");  
Qualification.setBounds(50,400,200,30);  
Qualification.setFont(new Font("Serif",Font.BOLD,20));  
add(Qualification);
```

```
String course[] = {"B.Tech", "BBA", "BCA",  
"BSC","MCA","MBA","B.COM"};  
courseBox = new JComboBox(course);  
courseBox.setBounds(200,400,150,30);  
courseBox.setBackground(Color.WHITE);  
add(courseBox);
```

```
JLabel Department = new JLabel("Department");
```

```
Department.setBounds(400,400,200,30);  
Department.setFont(new Font("Serif",Font.BOLD,20));  
add(Department);
```

```
String department[] = {"Computer  
Science","Electronics","Mechanical","Civil","IT"};  
departmentBox = new JComboBox(department);  
departmentBox.setBounds(600,400,150,30);  
departmentBox.setBackground(Color.WHITE);  
add(departmentBox);
```

```
submit = new JButton("Submit");  
submit.setBounds(250,550,120,30);  
submit.setBackground(Color.BLACK);  
submit.setForeground(Color.WHITE);  
submit.addActionListener(this);  
add(submit);
```

```
cancel = new JButton("Cancel");  
cancel.setBounds(450,550,120,30);  
cancel.setBackground(Color.BLACK);  
cancel.setForeground(Color.WHITE);  
cancel.addActionListener(this);  
add(cancel);
```

```
setSize(900,700);
```

```
    setLocation(350,50);
    setLayout(null);
    setVisible(true);
}
```

@Override

```
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == submit){
        String name = textName.getText();
        String fname = textfather.getText();
        String empid = empText.getText();
        String dob = ((JTextField)
cdob.getDateEditor().getUiComponent()).getText();
        String address = textAddress.getText();
        String phone = textPhone.getText();
        String email = textemail.getText();
        String x = textM10.getText();
        String xii = textM12.getText();
        String aadhar = textAadhar.getText();
        String course = (String) courseBox.getSelectedItem();
        String department = (String) departmentBox.getSelectedItem();
        try{
            String q = "insert into teacher
values('"+name+"','"+fname+"','"+empid+"','"+dob+"','"+address+"','"+phon
e+"','"+email+"','"+x+"','"+xii+"','"+aadhar+"','"+course+"','"+department+"')
";

            Conn c = new Conn();
            c.statement.executeUpdate(q);
```

```

        JOptionPane.showMessageDialog(null,"Details Inserted");
        setVisible(false);

    }catch (Exception E){
        E.printStackTrace();
    }
    }else{
        setVisible(false);
    }
}

public static void main(String[] args) {
    new AddFaculty();
}
}

```

ENTER MARKS

```

package university.management.system;

import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.ResultSet;

public class EnterMarks extends JFrame implements ActionListener {
    Choice choicerollno;

```

```
JComboBox comboBox;  
JTextField sub1, sub2, sub3, sub4, sub5, mrk1, mrk2, mrk3, mrk4, mrk5;  
JButton submit, cancel;
```

```
EnterMarks(){  
    getContentPane().setBackground(new Color(252,245,210));
```

```
    ImageIcon i1 = new  
ImageIcon(ClassLoader.getResource("icon/exam.png"));  
    Image i2 = i1.getImage().getScaledInstance(400,300,Image.SCALE_DEFAULT);  
    ImageIcon i3 = new ImageIcon(i2);  
    JLabel img = new JLabel(i3);  
    img.setBounds(500,40,400,300);  
    add(img);
```

```
JLabel heading = new JLabel("Enter Marks of Student");  
heading.setBounds(50,0,500,50);  
heading.setFont(new Font("Tahoma",Font.BOLD,20));  
add(heading);
```

```
JLabel rollno = new JLabel("Select Roll Number");  
rollno.setBounds(50,70,150,20);  
add(rollno);
```

```
choicerollno = new Choice();  
choicerollno.setBounds(200,70,150,20);  
add(choicerollno);
```

```
try{  
    Conn c = new Conn();  
    ResultSet resultSet = c.statement.executeQuery("select * from student");
```

```
        while(resultSet.next()){
            choicerollno.add(resultSet.getString("rollno"));
        }
    }catch (Exception e){
        e.printStackTrace();
    }
```

```
JLabel sem = new JLabel("Select Semester");
sem.setBounds(50,110,150,20);
add(sem);
```

```
String semester[] = {"1st Semester","2nd Semester","3rd Semester","4th
Semester","5th Semester","6th Semester","7th Semester","8th Semester"};
comboBox = new JComboBox(semester);
comboBox.setBounds(200,110,150,20);
comboBox.setBackground(Color.WHITE);
add(comboBox);
```

```
JLabel entersub = new JLabel("Enter Subject");
entersub.setBounds(100,150,200,40);
add(entersub);
```

```
JLabel entermarks = new JLabel("Enter Marks");
entermarks.setBounds(320,150,200,40);
add(entermarks);
```

```
sub1 = new JTextField();
sub1.setBounds(50,200,200,20);
add(sub1);
```

```
sub2 = new JTextField();
sub2.setBounds(50,230,200,20);
```

```
add(sub2);
```

```
sub3 = new JTextField();  
sub3.setBounds(50,260,200,20);  
add(sub3);
```

```
sub4 = new JTextField();  
sub4.setBounds(50,290,200,20);  
add(sub4);
```

```
sub5 = new JTextField();  
sub5.setBounds(50,320,200,20);  
add(sub5);
```

```
mrk1 = new JTextField();  
mrk1.setBounds(250,200,200,20);  
add(mrk1);
```

```
mrk2 = new JTextField();  
mrk2.setBounds(250,230,200,20);  
add(mrk2);
```

```
mrk3 = new JTextField();  
mrk3.setBounds(250,260,200,20);  
add(mrk3);
```

```
mrk4 = new JTextField();  
mrk4.setBounds(250,290,200,20);  
add(mrk4);
```

```
mrk5 = new JTextField();
```

```
mrk5.setBounds(250,320,200,20);  
add(mrk5);
```

```
submit = new JButton("Submit");  
submit.setBounds(70,360,150,25);  
submit.setBackground(Color.black);  
submit.setBackground(Color.WHITE);  
submit.addActionListener(this);  
add(submit);
```

```
cancel = new JButton("Cancel");  
cancel.setBounds(280,360,150,25);  
cancel.setBackground(Color.black);  
cancel.setBackground(Color.WHITE);  
cancel.addActionListener(this);  
add(cancel);  
setSize(1000,500);  
setLayout(null);  
setLocation(300,150);  
setVisible(true);  
}
```

@Override

```
public void actionPerformed(ActionEvent e) {  
    if(e.getSource() == submit){  
        try{  
            Conn c = new Conn();  
            String Q1 = "insert into subject  
values('"+choicerollno.getSelectedItemAt()+"', '"+comboBox.getSelectedItemAt()+"', '"+sub  
1.getText()+"', '"+sub2.getText()+"', '"+sub3.getText()+"', '"+sub4.getText()+"', '"+sub5.g  
etText()+"'");  
            String Q2 = "insert into marks
```



```
values(""+choicerollno.getSelectedItemId()+"", ""+comboBox.getSelectedItemId()+"", ""+mrk1.getText()+"", ""+mrk2.getText()+"", ""+mrk3.getText()+"", ""+mrk4.getText()+"", ""+mrk5.getText()+"");
```

```
        c.statement.executeUpdate(Q1);  
        c.statement.executeUpdate(Q2);  
        JOptionPane.showMessageDialog(null, "Marks Inserted Sucessfully");  
        setVisible(false);
```

```
    } catch (Exception E){  
        E.printStackTrace();  
    }  
}
```

```
}
```

```
public static void main(String[] args) {  
    new EnterMarks();  
}  
}
```

TEACHER DETAILS

```
package university.management.system;  
import net.proteanit.sql.DbUtils;  
import javax.swing.*;  
import java.awt.*;  
import java.awt.event.ActionEvent;
```

```

import java.awt.event.ActionListener;
import java.sql.ResultSet;

public class TeacherDetails extends JFrame implements ActionListener {

    Choice choice;
    JTable table;
    JButton search , print, add, cancel, update;

    TeacherDetails(){
        getContentPane().setBackground(new Color(192,164,252));

        JLabel heading = new JLabel("Search by Employee ID");
        heading.setBounds(20,20,150,20);
        add(heading);

        choice = new Choice();
        choice.setBounds(180,20,150,20);
        add(choice);

        try{
            Conn c = new Conn();
            ResultSet resultSet = c.statement.executeQuery("Select * from teacher");
            while (resultSet.next()){
                choice.add(resultSet.getString("empId"));
            }
        }catch (Exception e){
            e.printStackTrace();
        }

        table = new JTable();
        try{

```

```
Conn c = new Conn();
ResultSet resultSet = c.statement.executeQuery("select * from teacher");
table.setModel(DbUtils.resultSetToTableModel(resultSet));
}catch (Exception e){
    e.printStackTrace();
}
JScrollPane js = new JScrollPane(table);
js.setBounds(0,100,900,600);
add(js);
```

```
search = new JButton("Search");
search.setBounds(20,70,80,20);
search.addActionListener(this);
add(search);
```

```
print = new JButton("Print");
print.setBounds(120,70,80,20);
print.addActionListener(this);
add(print);
```

```
add = new JButton("Add");
add.setBounds(220,70,80,20);
add.addActionListener(this);
add(add);
```

```
update = new JButton("Update");
update.setBounds(320,70,80,20);
update.addActionListener(this);
add(update);
```

```
cancel = new JButton("Cancel");
cancel.setBounds(420,70,80,20);
```

```
cancel.addActionListener(this);  
add(cancel);
```

```
setLayout(null);  
setSize(900,700);  
setLocation(300,100);  
setVisible(true);  
}
```

```
@Override
```

```
public void actionPerformed(ActionEvent e) {  
    if (e.getSource() == search){  
        String q = "select * from teacher where empId  
= '"+choice.getSelectedItem()+"'";  
        try {  
            Conn c = new Conn();  
            ResultSet resultSet = c.statement.executeQuery(q);  
            table.setModel(DbUtils.resultSetToTableModel(resultSet));  
        }catch (Exception E){  
            E.printStackTrace();  
        }  
    }else if (e.getSource() == print){  
        try {  
            table.print();  
        }catch (Exception E){  
            E.printStackTrace();  
        }  
    }  
  
    } else if (e.getSource() == add) {  
        setVisible(false);  
        new AddFaculty();  
    }
```

```
    } else if (e.getSource() == update) {

        }else{
            setVisible(false);
        }
    }

    public static void main(String[] args) {
        new TeacherDetails();
    }
}
```

STUDENT DETAILS

```
package university.management.system;

import net.proteanit.sql.DbUtils;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.ResultSet;

public class StudentDetails extends JFrame implements ActionListener {
    Choice choice;
    JTable table;
```

JButton search , print, add, cancel, update;

```
StudentDetails(){
    getContentPane().setBackground(new Color(210,252,218));

    JLabel heading = new JLabel("Search by Roll Number");
    heading.setBounds(20,20,150,20);
    add(heading);

    choice = new Choice();
    choice.setBounds(180,20,150,20);
    add(choice);

    try{
        Conn c = new Conn();
        ResultSet resultSet = c.statement.executeQuery("Select * from student");
        while (resultSet.next()){
            choice.add(resultSet.getString("rollno"));
        }
    }catch (Exception e){
        e.printStackTrace();
    }

    table = new JTable();
    try{
        Conn c = new Conn();
        ResultSet resultSet = c.statement.executeQuery("select * from student");
        table.setModel(DbUtils.resultSetToTableModel(resultSet));
    }catch (Exception e){
        e.printStackTrace();
    }
    JScrollPane js = new JScrollPane(table);
```

```
js.setBounds(0,100,900,600);  
add(js);
```

```
search = new JButton("Search");  
search.setBounds(20,70,80,20);  
search.addActionListener(this);  
add(search);
```

```
print = new JButton("Print");  
print.setBounds(120,70,80,20);  
print.addActionListener(this);  
add(print);
```

```
add = new JButton("Add");  
add.setBounds(220,70,80,20);  
add.addActionListener(this);  
add(add);
```

```
update = new JButton("Update");  
update.setBounds(320,70,80,20);  
update.addActionListener(this);  
add(update);
```

```
cancel = new JButton("Cancel");  
cancel.setBounds(420,70,80,20);  
cancel.addActionListener(this);  
add(cancel);
```

```
setLayout(null);  
setSize(900,700);  
setLocation(300,100);  
setVisible(true);
```

```
}
```

```
@Override
```

```
public void actionPerformed(ActionEvent e) {  
    if (e.getSource() == search){  
        String q = "select * from student where rollno  
= '"+choice.getSelectedItem()+"'";  
        try {  
            Conn c = new Conn();  
            ResultSet resultSet = c.statement.executeQuery(q);  
            table.setModel(DbUtils.resultSetToTableModel(resultSet));  
        }catch (Exception E){  
            E.printStackTrace();  
        }  
    }else if (e.getSource() == print){  
        try {  
            table.print();  
        }catch (Exception E){  
            E.printStackTrace();  
        }  
    }  
  
    } else if (e.getSource() == add) {  
        setVisible(false);  
        new AddStudent();  
    }  
  
    } else if (e.getSource() == update) {  
  
    }else{  
        setVisible(false);  
    }  
}
```



```
}

public static void main(String[] args) {
    new StudentDetails();
}
}
```

FEE STRUCTURE

```
package university.management.system;

import net.proteanit.sql.DbUtils;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.ResultSet;

public class FeeStructure extends JFrame implements ActionListener {

    FeeStructure(){
        getContentPane().setBackground(Color.WHITE);
        JLabel heading = new JLabel("Fee Structure");
        heading.setBounds(400,10,400,30);
        heading.setFont(new Font("Tahoma",Font.BOLD,30));
        add(heading);
    }
}
```

```
JTable table = new JTable();
```

```
try{
```

```
    Conn c = new Conn();
```

```
    ResultSet resultSet = c.statement.executeQuery("select * from fee");
```

```
    table.setModel(DbUtils.resultSetToTableModel(resultSet));
```

```
}catch (Exception e){
```

```
    e.printStackTrace();
```

```
}
```

```
JScrollPane js = new JScrollPane(table);
```

```
js.setBounds(0,60,1000,700);
```

```
add(js);
```

```
setSize(1000,700);
```

```
setLocation(250,50);
```

```
setLayout(null);
```

```
setVisible(true);
```

```
}
```

```
@Override
```

```
public void actionPerformed(ActionEvent e) {
```

```
}
```

```
public static void main(String[] args) {
```

```
    new FeeStructure();
```

```
}
```

```
}
```

EXAMINATION DETAILS

```
package university.management.system;
```

```
import net.proteanit.sql.DbUtils;
```

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
import java.sql.ResultSet;
```

```
public class ExaminationDetails extends JFrame implements ActionListener {
```

```
    JTextField search;
```

```
    JButton result, back;
```

```
    JTable table;
```

```
    ExaminationDetails(){
```

```
        getContentPane().setBackground(new Color(241,252,210));
```

```
        JLabel heading = new JLabel("Check Result");
```

```
        heading.setBounds(350,15,400,50);
```

```
        heading.setFont(new Font("Tahoma",Font.BOLD,24));
```

```
        add(heading);
```

```
        search = new JTextField();
```

```
        search.setBounds(80,90,200,30);
```

```
search.setFont(new Font("Tahoma", Font.PLAIN,18));  
add(search);
```

```
result = new JButton("Result");  
result.setBounds(300,90,120,30);  
result.setBackground(Color.black);  
result.setForeground(Color.WHITE);  
result.addActionListener(this);  
add(result);
```

```
back = new JButton("Back");  
back.setBounds(440,90,120,30);  
back.setBackground(Color.black);  
back.setForeground(Color.WHITE);  
back.addActionListener(this);  
add(back);
```

```
table = new JTable();  
JScrollPane scrollPane = new JScrollPane(table);  
scrollPane.setBounds(0,130,1000,310);  
add(scrollPane);
```

```
try{  
    Conn c = new Conn();  
    ResultSet resultSet = c.statement.executeQuery("select * from student");  
    table.setModel(DbUtils.resultSetToTableModel(resultSet));  
}catch (Exception e){  
    e.printStackTrace();  
}
```

```
table.addMouseListener(new MouseAdapter() {
    @Override
    public void mouseClicked(MouseEvent e) {
        int row = table.getSelectedRow();
        search.setText(table.getModel().getValueAt(row,2).toString());
    }
});
```

```
setSize(1000,475);
setLocation(300,100);
setLayout(null);
setVisible(true);
}
```

```
@Override
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == result){
        setVisible(false);
        new Marks(search.getText());
    }else {
        setVisible(false);
    }
}
```

```
public static void main(String[] args) {
    new ExaminationDetails();
}
}
```

STUDENT LEAVE DETAILS

```
package university.management.system;
```

```
import net.proteanit.sql.DbUtils;
```

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
import java.awt.event.ActionEvent;
```

```
import java.awt.event.ActionListener;
```

```
import java.sql.ResultSet;
```

```
public class StudentLeaveDetails extends JFrame implements ActionListener {
```

```
    Choice choiceEmpID;
```

```
    JTable table;
```

```
    JButton search, cancel , print;
```

```
    StudentLeaveDetails(){
```

```
        getContentPane().setBackground(new Color(250,172,206));
```

```
        JLabel heading = new JLabel("Search by Roll Number");
```

```
        heading.setBounds(20,20,150,20);
```

```
        add(heading);
```

```
        choiceEmpID = new Choice();
```

```
choiceEmpID.setBounds(180,20,150,20);  
add(choiceEmpID);
```

```
try {  
    Conn c = new Conn();  
    ResultSet resultSet = c.statement.executeQuery("select * from student");  
    while (resultSet.next()){  
        choiceEmpID.add(resultSet.getString("rollno"));  
    }  
}catch (Exception e){  
    e.printStackTrace();  
}
```

```
table = new JTable();  
try {  
    Conn c = new Conn();  
    ResultSet resultSet = c.statement.executeQuery("select * from studentleave");  
    table.setModel(DbUtils.resultSetToTableModel(resultSet));  
}catch (Exception e){  
    e.printStackTrace();  
}
```

```
JScrollPane scrollPane = new JScrollPane(table);  
scrollPane.setBounds(0,100,900,600);  
add(scrollPane);
```

```
search = new JButton("Search");  
search.setBounds(20,70,80,20);  
search.addActionListener(this);  
add(search);
```

```
print = new JButton("Print");  
print.setBounds(120,70,80,20);
```

```
print.addActionListener(this);  
add(print);
```

```
cancel = new JButton("Cancel");  
cancel.setBounds(220,70,80,20);  
cancel.addActionListener(this);  
add(cancel);
```

```
setSize(900,700);  
setLocation(300,100);  
setLayout(null);  
setVisible(true);  
}
```

@Override

```
public void actionPerformed(ActionEvent e) {  
    if(e.getSource() == search){  
        String q = "select * from studentleave where rollno =  
"+choiceEmpID.getSelectedItem()+"";  
        try{  
            Conn c = new Conn();  
            ResultSet resultSet = c.statement.executeQuery(q);  
            table.setModel(DbUtils.resultSetToTableModel(resultSet));  
        }catch (Exception E){  
            E.printStackTrace();  
        }  
    } else if (e.getSource() == print) {  
        try {  
            table.print();  
        }catch (Exception E){  
            E.printStackTrace();  
        }  
    }  
}
```



```

    }else{
        setVisible(false);
    }

}

public static void main(String[] args) {
    new StudentLeaveDetails();
}
}

```

TEACHER LEAVE DETAILS

```
package university.management.system;
```

```
import com.mysql.cj.util.DnsSrv;
import net.proteanit.sql.DbUtils;
```

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.ResultSet;
```

```
public class TeacherLeaveDetails extends JFrame implements ActionListener {
    Choice choiceEmpID;
    JTable table;
    JButton search, cancel , print;

    TeacherLeaveDetails(){

```

```
getContentPane().setBackground(new Color(250,172,206));
```

```
JLabel heading = new JLabel("Search by Employee Id");  
heading.setBounds(20,20,150,20);  
add(heading);
```

```
choiceEmpID = new Choice();  
choiceEmpID.setBounds(180,20,150,20);  
add(choiceEmpID);
```

```
try {  
    Conn c = new Conn();  
    ResultSet resultSet = c.statement.executeQuery("select * from teacher");  
    while (resultSet.next()){  
        choiceEmpID.add(resultSet.getString("empId"));  
    }  
}catch (Exception e){  
    e.printStackTrace();  
}
```

```
table = new JTable();  
try {  
    Conn c = new Conn();  
    ResultSet resultSet = c.statement.executeQuery("select * from  
teacherleave");  
    table.setModel(DbUtils.resultSetToTableModel(resultSet));  
}catch (Exception e){  
    e.printStackTrace();  
}
```

```
JScrollPane scrollPane = new JScrollPane(table);  
scrollPane.setBounds(0,100,900,600);
```

```
add(scrollPane);
```

```
search = new JButton("Search");  
search.setBounds(20,70,80,20);  
search.addActionListener(this);  
add(search);
```

```
print = new JButton("Print");  
print.setBounds(120,70,80,20);  
print.addActionListener(this);  
add(print);
```

```
cancel = new JButton("Cancel");  
cancel.setBounds(220,70,80,20);  
cancel.addActionListener(this);  
add(cancel);
```

```
setSize(900,700);  
setLocation(300,100);  
setLayout(null);  
setVisible(true);
```

```
}
```

```
@Override
```

```
public void actionPerformed(ActionEvent e) {  
    if(e.getSource() == search){  
        String q = "select * from teacherleave where empld =  
        '"+choiceEmpID.getSelectedItem()+"';  
        try{  
            Conn c = new Conn();
```

```

        ResultSet resultSet = c.statement.executeQuery(q);
        table.setModel(DbUtils.resultSetToTableModel(resultSet));
    }catch (Exception E){
        E.printStackTrace();
    }
} else if (e.getSource() == print) {
    try {
        table.print();
    }catch (Exception E){
        E.printStackTrace();
    }

} else {
    setVisible(false);
}

}

public static void main(String[] args) {
    new TeacherLeaveDetails();
}
}

```

UPDATE STUDENT RECORD

```

package university.management.system;

import javax.swing.*.*;
import java.awt.*.*;

```

```

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;
import java.sql.ResultSet;

public class UpdateStudent extends JFrame implements ActionListener {
    JTextField textAddress, textPhone, textemail, textAadhar, textcourse,
    textbranch;
    JLabel empText;
    JButton submit, cancel;
    Choice cEMPID;

    UpdateStudent(){
        getContentPane().setBackground(new Color(230,210,252));

        JLabel heading = new JLabel("Update Student Details");
        heading.setBounds(50,10,500,50);
        heading.setFont(new Font("serif",Font.BOLD,35));
        add(heading);

        JLabel empID = new JLabel("Select Roll Number ");
        empID.setBounds(50,100,200,20);
        empID.setFont(new Font("serif", Font.PLAIN,20));
        add(empID);

        cEMPID =new Choice();
        cEMPID.setBounds(250,100,200,20);
        add(cEMPID);

        try{
            Conn c = new Conn();

```

```
ResultSet rs = c.statement.executeQuery("select * from student");
while (rs.next()){
    cEMPID.add(rs.getString("rollno"));
}
}catch (Exception e){
    e.printStackTrace();
}
```

```
JLabel name = new JLabel("Name");
name.setBounds(50,150,100,30);
name.setFont(new Font("Serif",Font.BOLD,20));
add(name);
```

```
JLabel textName = new JLabel();
textName.setBounds(200,150,150,30);
add(textName);
```

```
JLabel fname = new JLabel("Father Name");
fname.setBounds(400,150,200,30);
fname.setFont(new Font("Serif",Font.BOLD,20));
add(fname);
```

```
JLabel textfather = new JLabel();
textfather.setBounds(600,150,150,30);
add(textfather);
```

```
JLabel EMPIDD = new JLabel("Roll Number");
EMPIDD.setBounds(50,200,200,30);
EMPIDD.setFont(new Font("Serif",Font.BOLD,20));
add(EMPIDD);
```

```
empText = new JLabel();  
empText.setBounds(200,200,150,30);  
empText.setFont(new Font("Serif",Font.BOLD,20));  
add(empText);
```

```
JLabel dob = new JLabel("Date Of Birth");  
dob.setBounds(400,200,200,30);  
dob.setFont(new Font("Serif",Font.BOLD,20));  
add(dob);
```

```
JLabel dobdob = new JLabel();  
dobdob.setBounds(600,200,150,30);  
add(dobdob);
```

```
JLabel address = new JLabel("Address");  
address.setBounds(50,250,200,30);  
address.setFont(new Font("Serif",Font.BOLD,20));  
add(address);
```

```
textAddress = new JTextField();  
textAddress.setBounds(200,250,150,30);  
add(textAddress);
```

```
JLabel phone = new JLabel("Phone");  
phone.setBounds(400,250,200,30);  
phone.setFont(new Font("Serif",Font.BOLD,20));  
add(phone);
```

```
textPhone = new JTextField();  
textPhone.setBounds(600,250,150,30);  
add(textPhone);
```

```
JLabel email = new JLabel("Email");  
email.setBounds(50,300,200,30);  
email.setFont(new Font("Serif",Font.BOLD,20));  
add(email);
```

```
textemail = new JTextField();  
textemail.setBounds(200,300,150,30);  
add(textemail);
```

```
JLabel M10 = new JLabel("Class X (%)");  
M10.setBounds(400,300,200,30);  
M10.setFont(new Font("Serif",Font.BOLD,20));  
add(M10);
```

```
JLabel textM10 = new JLabel();  
textM10.setBounds(600,300,150,30);  
add(textM10);
```

```
JLabel M12 = new JLabel("Class XII (%)");  
M12.setBounds(50,350,200,30);  
M12.setFont(new Font("Serif",Font.BOLD,20));  
add(M12);
```

```
JLabel textM12 = new JLabel();  
textM12.setBounds(200,350,150,30);  
add(textM12);
```



```
JLabel AadharNo = new JLabel("Aadhar Number");  
AadharNo.setBounds(400,350,200,30);  
AadharNo.setFont(new Font("Serif",Font.BOLD,20));  
add(AadharNo);
```

```
textAadhar = new JTextField();  
textAadhar.setBounds(600,350,150,30);  
add(textAadhar);
```

```
JLabel Qualification = new JLabel("Course");  
Qualification.setBounds(50,400,200,30);  
Qualification.setFont(new Font("Serif",Font.BOLD,20));  
add(Qualification);
```

```
textcourse = new JTextField();  
textcourse.setBounds(200,400,150,30);  
add(textcourse);
```

```
JLabel Department = new JLabel("Branch");  
Department.setBounds(400,400,200,30);  
Department.setFont(new Font("Serif",Font.BOLD,20));  
add(Department);
```

```
textbranch = new JTextField();  
textbranch.setBounds(600,400,150,30);  
add(textbranch);
```

```
try {  
    Conn c = new Conn();  
    String query = "select * from student where rollno =
```

```

""+cEMPID.getSelectedItem()+"";
    ResultSet resultSet = c.statement.executeQuery(query);
    while (resultSet.next()){
        textName.setText(resultSet.getString("name"));
        textfather.setText(resultSet.getString("fname"));
        dobdob.setText(resultSet.getString("dob"));
        textAddress.setText(resultSet.getString("address"));
        textPhone.setText(resultSet.getString("phone"));
        textemail.setText(resultSet.getString("email"));
        textM10.setText(resultSet.getString("class_x"));
        textM12.setText(resultSet.getString("class_xii"));
        textAadhar.setText(resultSet.getString("aadhar"));
        empText.setText(resultSet.getString("rollno"));
        textcourse.setText(resultSet.getString("course"));
        textbranch.setText(resultSet.getString("branch"));
    }
} catch (Exception E){
    E.printStackTrace();
}

cEMPID.addItemListener(new ItemListener() {
    @Override
    public void itemStateChanged(ItemEvent e) {
        try {
            Conn c = new Conn();
            String query = "select * from student where rollno =
""+cEMPID.getSelectedItem()+"";
            ResultSet resultSet = c.statement.executeQuery(query);
            while (resultSet.next()) {
                textName.setText(resultSet.getString("name"));
                textfather.setText(resultSet.getString("fname"));
                dobdob.setText(resultSet.getString("dob"));

```

```
        textAddress.setText(resultSet.getString("address"));
        textPhone.setText(resultSet.getString("phone"));
        textemail.setText(resultSet.getString("email"));
        textM10.setText(resultSet.getString("class_x"));
        textM12.setText(resultSet.getString("class_xii"));
        textAadhar.setText(resultSet.getString("aadhar"));
        empText.setText(resultSet.getString("rollno"));
        textcourse.setText(resultSet.getString("course"));
        textbranch.setText(resultSet.getString("branch"));
    }
} catch (Exception E){
    E.printStackTrace();

}
}
});
```

```
submit = new JButton("Update");
submit.setBounds(250,550,120,30);
submit.setBackground(Color.BLACK);
submit.setForeground(Color.WHITE);
submit.addActionListener(this);
add(submit);
```

```
cancel = new JButton("Cancel");
cancel.setBounds(450,550,120,30);
cancel.setBackground(Color.BLACK);
cancel.setForeground(Color.WHITE);
cancel.addActionListener(this);
add(cancel);
```

```
setSize(900,700);
setLocation(350,50);
setLayout(null);
setVisible(true);
}
```

@Override

```
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == submit){
        String empid = empText.getText();
        String address = textAddress.getText();
        String phone = textPhone.getText();
        String email = textemail.getText();
        String course = textcourse.getText();
        String branch = textbranch.getText();

        try{
            String Q = "update student set address = '"+address+"',phone = 
            '"+phone+"', email = '"+email+"', course = '"+course+"', branch = '"+branch+" 
            where rollno = '"+empid+"'";
            Conn c = new Conn();
            c.statement.executeUpdate(Q);

            JOptionPane.showMessageDialog(null,"Details Updated");
            setVisible(false);

        }catch (Exception E){
            E.printStackTrace();
        }
    }
}
```

```
        }else{
            setVisible(false);
        }
    }

    public static void main(String[] args) {
        new UpdateStudent();
    }
}
```

UPDATE TEACHER RECORD

```
package university.management.system;

import com.mysql.cj.util.DnsSrv;
import com.toedter.calendar.JDateChooser;

import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;
import java.sql.ResultSet;

public class UpdateTeacher extends JFrame implements ActionListener
```

```
{
    JTextField textAddress, textPhone, textemail, textAadhar, textcourse,
    textbranch;
    JLabel empText;
    JButton submit, cancel;
    Choice cEMPID;
```

```
UpdateTeacher(){
```

```
    getContentPane().setBackground(new Color(230,210,252));
```

```
    JLabel heading = new JLabel("Update Teacher Details");
    heading.setBounds(50,10,500,50);
    heading.setFont(new Font("serif",Font.BOLD,35));
    add(heading);
```

```
    JLabel empID = new JLabel("Select Employee Id");
    empID.setBounds(50,100,200,20);
    empID.setFont(new Font("serif", Font.PLAIN,20));
    add(empID);
```

```
    cEMPID =new Choice();
    cEMPID.setBounds(250,100,200,20);
    add(cEMPID);
```

```
    try{
```

```
Conn c = new Conn();
ResultSet rs = c.statement.executeQuery("select * from
teacher");
while (rs.next()){
    cEMPID.add(rs.getString("empId"));
}
}catch (Exception e){
    e.printStackTrace();
}
```

```
JLabel name = new JLabel("Name");
name.setBounds(50,150,100,30);
name.setFont(new Font("Serif",Font.BOLD,20));
add(name);
```

```
JLabel textName = new JLabel();
textName.setBounds(200,150,150,30);
add(textName);
```

```
JLabel fname = new JLabel("Father Name");
fname.setBounds(400,150,200,30);
fname.setFont(new Font("Serif",Font.BOLD,20));
add(fname);
```

```
JLabel textfather = new JLabel();
textfather.setBounds(600,150,150,30);
add(textfather);
```

```
JLabel EMPIDD = new JLabel("Employee ID");  
EMPIDD.setBounds(50,200,200,30);  
EMPIDD.setFont(new Font("Serif",Font.BOLD,20));  
add(EMPIDD);
```

```
empText = new JLabel();  
empText.setBounds(200,200,150,30);  
empText.setFont(new Font("Serif",Font.BOLD,20));  
add(empText);
```

```
JLabel dob = new JLabel("Date Of Birth");  
dob.setBounds(400,200,200,30);  
dob.setFont(new Font("Serif",Font.BOLD,20));  
add(dob);
```

```
JLabel dobdob = new JLabel();  
dobdob.setBounds(600,200,150,30);  
add(dobdob);
```

```
JLabel address = new JLabel("Address");  
address.setBounds(50,250,200,30);  
address.setFont(new Font("Serif",Font.BOLD,20));  
add(address);
```

```
textAddress = new JTextField();  
textAddress.setBounds(200,250,150,30);
```



```
add(textAddress);
```

```
JLabel phone = new JLabel("Phone");  
phone.setBounds(400,250,200,30);  
phone.setFont(new Font("Serif",Font.BOLD,20));  
add(phone);
```

```
textPhone = new JTextField();  
textPhone.setBounds(600,250,150,30);  
add(textPhone);
```

```
JLabel email = new JLabel("Email");  
email.setBounds(50,300,200,30);  
email.setFont(new Font("Serif",Font.BOLD,20));  
add(email);
```

```
textemail = new JTextField();  
textemail.setBounds(200,300,150,30);  
add(textemail);
```

```
JLabel M10 = new JLabel("Class X (%)");  
M10.setBounds(400,300,200,30);  
M10.setFont(new Font("Serif",Font.BOLD,20));  
add(M10);
```

```
JLabel textM10 = new JLabel();
```

```
textM10.setBounds(600,300,150,30);  
add(textM10);
```

```
JLabel M12 = new JLabel("Class XII (%)");  
M12.setBounds(50,350,200,30);  
M12.setFont(new Font("Serif",Font.BOLD,20));  
add(M12);
```

```
JLabel textM12 = new JLabel();  
textM12.setBounds(200,350,150,30);  
add(textM12);
```

```
JLabel AadharNo = new JLabel("Aadhar Number");  
AadharNo.setBounds(400,350,200,30);  
AadharNo.setFont(new Font("Serif",Font.BOLD,20));  
add(AadharNo);
```

```
textAadhar = new JTextField();  
textAadhar.setBounds(600,350,150,30);  
add(textAadhar);
```

```
JLabel Qualification = new JLabel("Qualification");  
Qualification.setBounds(50,400,200,30);  
Qualification.setFont(new Font("Serif",Font.BOLD,20));  
add(Qualification);
```

```
textcourse = new JTextField();
textcourse.setBounds(200,400,150,30);
add(textcourse);
```

```
JLabel Department = new JLabel("Department");
Department.setBounds(400,400,200,30);
Department.setFont(new Font("Serif",Font.BOLD,20));
add(Department);
```

```
textbranch = new JTextField();
textbranch.setBounds(600,400,150,30);
add(textbranch);
```

```
try {
    Conn c = new Conn();
    String query = "select * from teacher where empld =
"+cEMPID.getSelectedItem()+"";
    ResultSet resultSet = c.statement.executeQuery(query);
    while (resultSet.next()){
        textName.setText(resultSet.getString("name"));
        textfather.setText(resultSet.getString("fname"));
        dobdob.setText(resultSet.getString("dob"));
        textAddress.setText(resultSet.getString("address"));
        textPhone.setText(resultSet.getString("phone"));
        textemail.setText(resultSet.getString("email"));
        textM10.setText(resultSet.getString("class_x"));
    }
}
```

```

        textM12.setText(resultSet.getString("class_xii"));
        textAadhar.setText(resultSet.getString("aadhar"));
        empText.setText(resultSet.getString("empId"));
        textcourse.setText(resultSet.getString("education"));
        textbranch.setText(resultSet.getString("department"));
    }
} catch (Exception E){
    E.printStackTrace();
}

cEMPID.addItemListener(new ItemListener() {
    @Override
    public void itemStateChanged(ItemEvent e) {
        try {
            Conn c = new Conn();
            String query = "select * from teacher where empId =
"+cEMPID.getSelectedItem()+"";
            ResultSet resultSet = c.statement.executeQuery(query);
            while (resultSet.next()) {
                textName.setText(resultSet.getString("name"));
                textfather.setText(resultSet.getString("fname"));
                dobdob.setText(resultSet.getString("dob"));
                textAddress.setText(resultSet.getString("address"));
                textPhone.setText(resultSet.getString("phone"));
                textemail.setText(resultSet.getString("email"));
                textM10.setText(resultSet.getString("class_x"));
                textM12.setText(resultSet.getString("class_xii"));
                textAadhar.setText(resultSet.getString("aadhar"));
            }
        } catch (Exception E){
            E.printStackTrace();
        }
    }
});

```

```
        empText.setText(resultSet.getString("empId"));
        textcourse.setText(resultSet.getString("education"));
        textbranch.setText(resultSet.getString("department"));
    }
} catch (Exception E){
    E.printStackTrace();

}
});
```

```
submit = new JButton("Update");
submit.setBounds(250,550,120,30);
submit.setBackground(Color.BLACK);
submit.setForeground(Color.WHITE);
submit.addActionListener(this);
add(submit);
```

```
cancel = new JButton("Cancel");
cancel.setBounds(450,550,120,30);
cancel.setBackground(Color.BLACK);
cancel.setForeground(Color.WHITE);
cancel.addActionListener(this);
add(cancel);
```

```
setSize(900,700);
```

```
    setLocation(350,50);  
    setLayout(null);  
    setVisible(true);  
}
```

@Override

```
public void actionPerformed(ActionEvent e) {  
    if (e.getSource() == submit){  
        String empid = empText.getText();  
        String address = textAddress.getText();  
        String phone = textPhone.getText();  
        String email = textemail.getText();  
        String course = textcourse.getText();  
        String branch = textbranch.getText();  
  
        try{  
            String Q = "update teacher set address = '"+address+"',phone =  
            '"+phone+"', email = '"+email+"', eduction = '"+course+"', department  
            = '"+branch+"' where empId = '"+empid+"'";  
            Conn c = new Conn();  
            c.statement.executeUpdate(Q);  
  
            JOptionPane.showMessageDialog(null,"Details Updated");  
            setVisible(false);  
  
        }catch (Exception E){  
            E.printStackTrace();  
        }  
    }  
}
```

```
    }  
    }else{  
        setVisible(false);  
    }  
  
}  
  
public static void main(String[] args) {  
    new UpdateTeacher();  
}  
}
```

TESTING

System testing is the stage of implementation, which is aimed at ensuring that the system works accurately and efficiently before live operation commences. Testing is the process of executing the program with the intent of finding errors and missing operations and also a complete verification to determine whether the objectives are met and the user requirements are satisfied. The ultimate aim is quality assurance.

Unit Testing

The software units in a system are modules and routines that are assembled and integrated to perform a specific function. Unit testing focuses first on modules, independently of one another, to locate errors. This enables, to detect errors in coding and logic that are contained within each module. This testing includes entering data and ascertaining if the value matches to the type and size supported by java. The various controls are tested to ensure that each performs its action as required.

Integration Testing

Data can be lost across any interface, one module can have an adverse effect on another, sub functions when combined, may not produce the desired major functions. Integration testing is a systematic testing to discover errors associated within the interface. The objective is to take unit tested modules and build a program structure. All the modules are combined and tested as a whole. Here the Server module and Client module options are integrated and tested. This testing provides the assurance that the application is well integrated functional unit with smooth transition of data.

User Acceptance

Testing User acceptance of a system is the key factor for the success of any system. The system under consideration is tested for user acceptance by constantly keeping in touch with the system users at time of developing and making changes whenever required.

Test Cases:

Table Test cases

Test No.	Test Name	input	Actual output	Expected output	Status
1	Login	Username and password	User is successfully Authenticated	User is successfully Authenticated	Pass
2	Login	Wrong username and password	Invalid username or password	Invalid username or password	Pass
3	Signup	User details and password	Account successfully created	Account successfully created	Pass
4	Student	Details of the student required.	Student inserted successfully	Student inserted successfully	Pass
5	Teacher	Details of the teacher required	Teacher inserted successfully	Teacher inserted successfully	Pass
6	Subject	Enter the subject names and marks along with rollno	Subjects entered successfully	Subjects entered successfully	Pass
7	Fee	Details and fee_paid	Paid successfully	Paid successfully	Pass

8	Remove Student	Enter rollno and click on remove	Removed successfully	Removed successfully	Pass
9	Remove Teacher	Enter emp_id and click on remove	Teacher removed successfully	Teacher removed successfully	Pass
10	Exit	Click on Exit	Logout successfully	Logout successfully	Pass

SNAPSHOTS

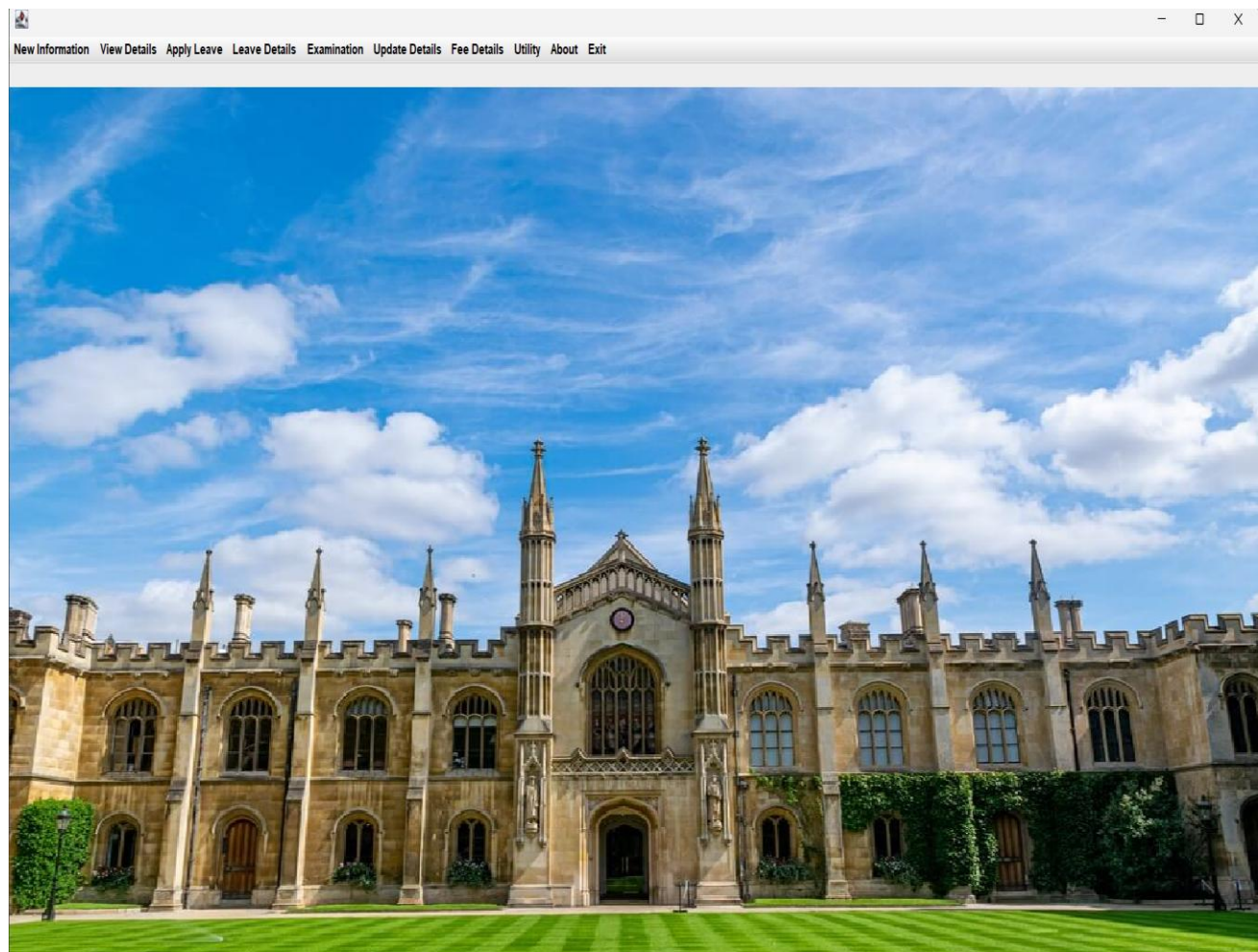
Login form :

This page represents the first thing about our website. It leads on to the login point for its personnel; it takes up the username, password and signup.

Login form

Home Page User :

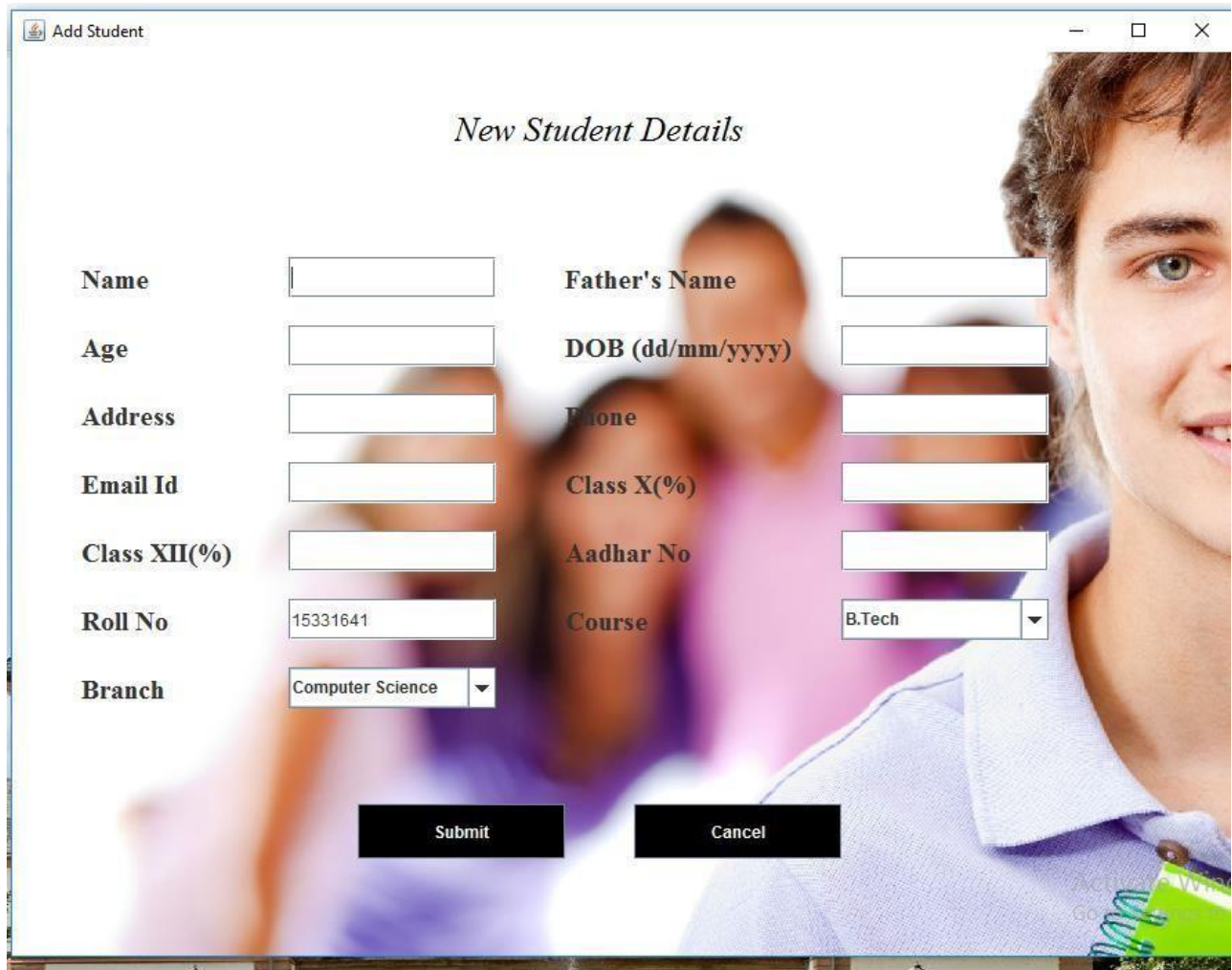
This page shows us what user can see and access. He can add, remove, update and upload the data. He can logout from the website in homepage.



Home page user

Student form :

In this we can add the new student details which will be stored in back end of user. This details further can updated in the update page.



Add Student

New Student Details

Name		Father's Name	
Age		DOB (dd/mm/yyyy)	
Address		Phone	
Email Id		Class X(%)	
Class XII(%)		Aadhar No	
Roll No	15331641	Course	B.Tech
Branch	Computer Science		

Submit **Cancel**

Student form

Teacher form:

In this we can add the new teacher details which will be stored in back end of user. This details further can updated in the update page.



The screenshot shows a web browser window with a light blue background. The title of the window is "New Teacher Details". The form contains the following fields:

Field Name	Value / Selection
Name	
Employee ID	14096231
Address	
Email	
Class XII (%)	
Qualification	B.Tech (dropdown)
Father Name	
Date Of Birth	(calendar icon)
Phone	
Class X (%)	
Aadhar Number	
Department	Computer Science (dropdown)

At the bottom of the form, there are two buttons: "Submit" and "Cancel".

Teacher form

Marks and Subject page :

In this page we can enter the subjects and marks scored in that particular subject along the roll no.

Marks and Subject page

Fee payment page :

In this page we can pay the fee dues of the particular student which uses rollno, course, branch and sem to pay the fee.

Fee Payment page

CONCLUSION

The project entitled as Institution Management System is the system that deals with the issues related to a particular institution

This project is successfully implemented with all the features mentioned in system requirements specification.

The application provides appropriate information to users according to the chosen service.

The project is designed keeping in view the day to day problems faced by a college.

Deployment of our application will certainly help the college to reduce unnecessary wastage of time in personally going to each department for some information.

Awareness and right information about any college is essential for both the development of student as well as

faculty. So this serves the right purpose in achieving the desired requirements of both the communities.

REFERENCES

Books and Websites:

Internet & World Wide Web: How to Program Deitel, PJ Deitel. Code for Interview YouTube Channel.

Database System Concepts, by Silberschatz, Sudarshan, and Korth. Fundamentals of Database Systems, Ramez Elmasri and Shamkant B. Navathe, 7th Edition 2017, Pearson.

